

Ansible Linux Automation Workshop

Introduction to Ansible for Red Hat Enterprise Linux Automation
for System Administrators and Operators

What you will learn

- ▶ Intro to Ansible Automation Platform
- ▶ How Ansible Works
- ▶ Understanding Modules, Tasks, Playbooks
- ▶ Leveraging Variables & Templates for Flexibility
- ▶ Automation Controller: It's Role in the Automation Ecosystem
- ▶ Automation Controller Basics & Key Concepts
- ▶ Core Features of Automation Controller: RBAC, Workflows



Introduction

Topics Covered:

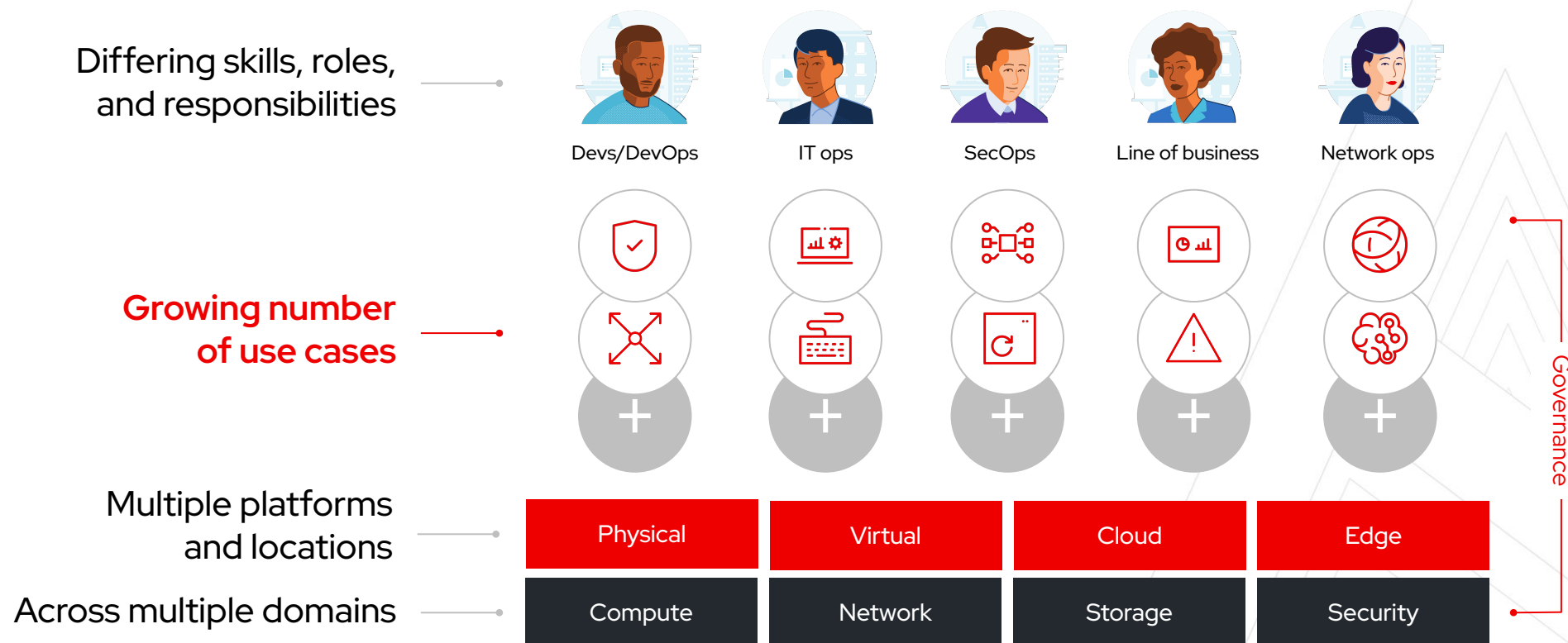
- Why the Ansible Automation Platform?
- What can it do?



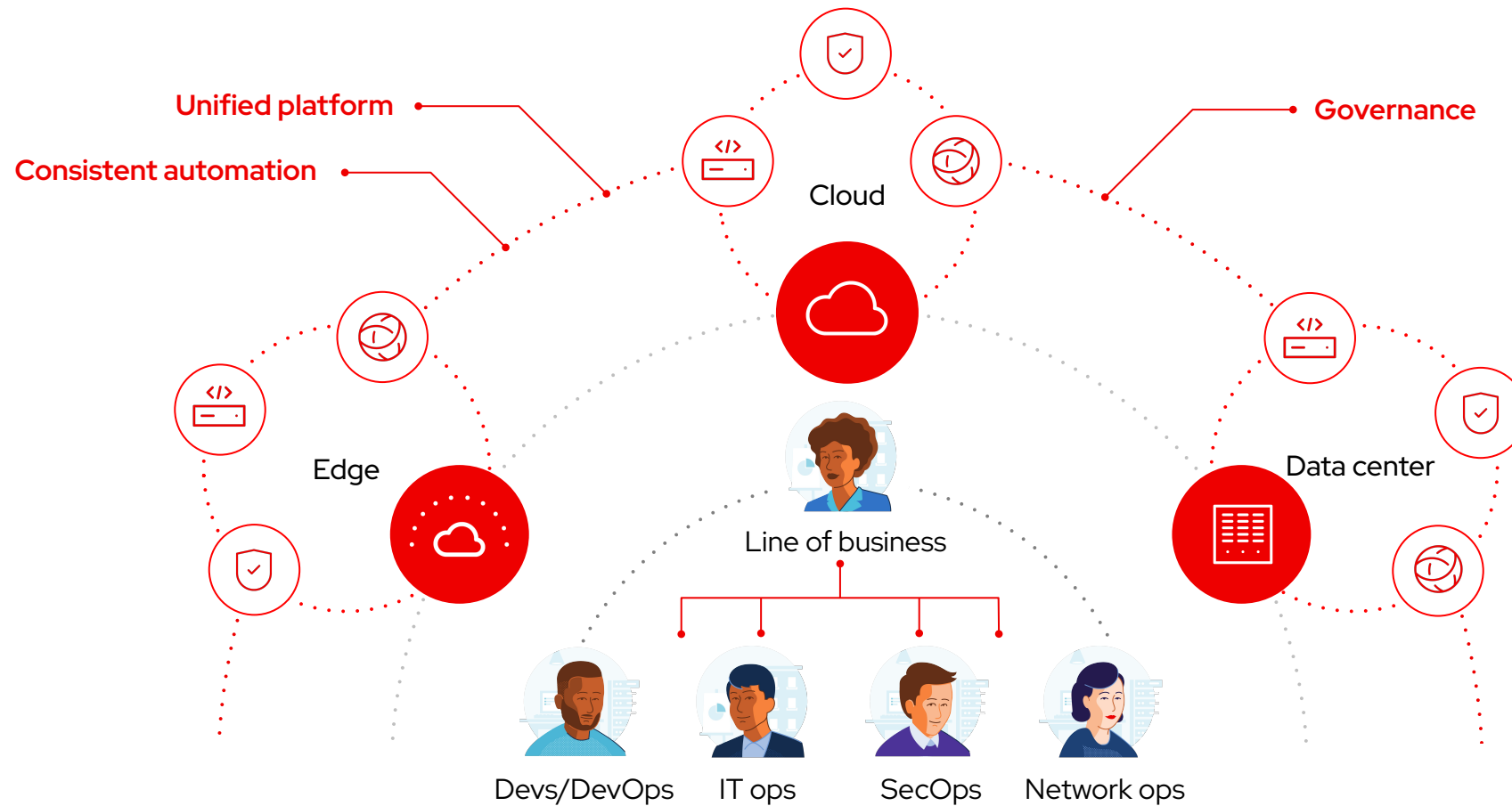


An enterprise needs to unlock its
automation advantage

Many organizations **share the same challenge.**



The solution? Break down the silos.



Why Ansible Automation Platform?

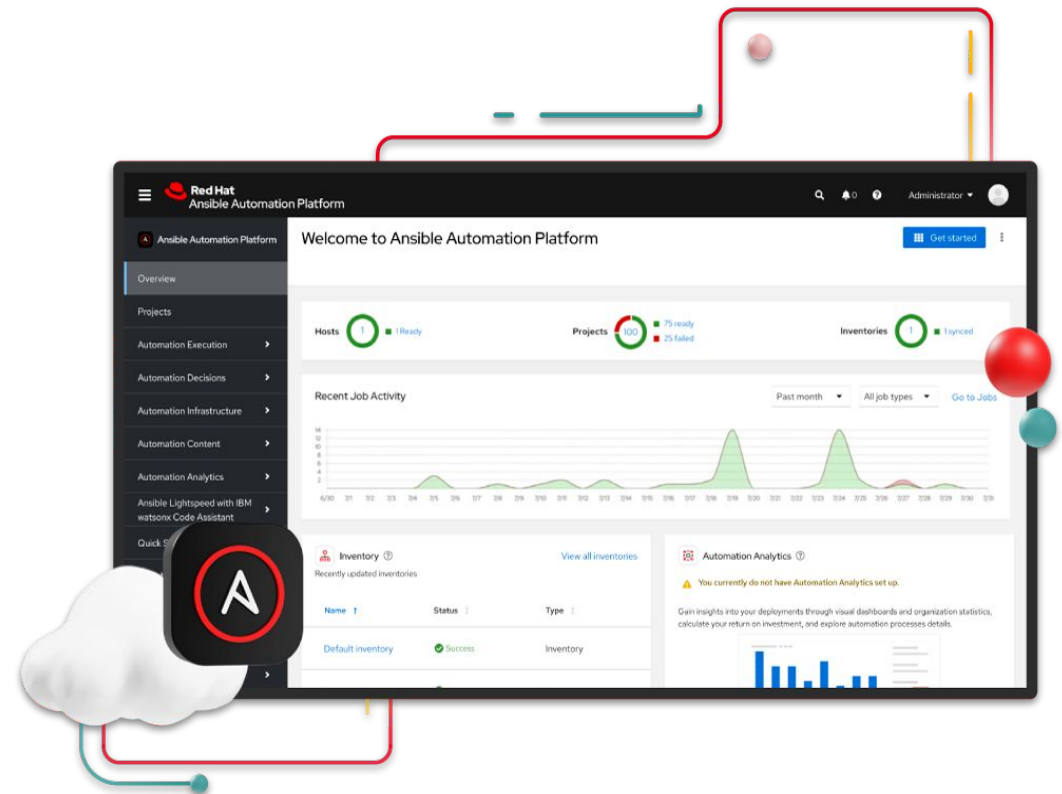
To unlock your automation advantage

Red Hat Ansible is becoming increasingly mission-critical for customers who rely on automation to bridge skills gaps, tame operational complexity across the enterprise, and mitigate costly tool sprawl.

Our goal with Red Hat Ansible Automation Platform 2.6 is to make it easier for every customer to fully unlock the potential of automation to transform IT – and deliver strategic advantages to the business.

This latest release is engineered to help our customers:

- **Accelerate automation adoption at scale**, with a reimagined automation platform experience, new features for enhanced usability, and tools for more effective collaboration and coordination.
- **Empower automation engineers**, with integrated developer tooling and generative AI capabilities designed to bolster ease and efficiency for a range of skill sets and experience across the entire automation creation lifecycle.
- **Orchestrate across the enterprise**, with event-driven automation capabilities that make intelligence from other tools more actionable, along with a robust ecosystem of integrations that put true end-to-end automation within reach.



Automate the deployment and management of automation

Your entire IT footprint

Do this...

Orchestrate

Manage configurations

Deploy applications

Provision / deprovision

Deliver continuously

Secure and comply

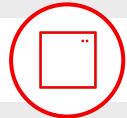
On these...



Firewalls



Load balancers



Applications



Containers



Virtualization platforms



Servers



Clouds



Storage



Network devices



And more ...

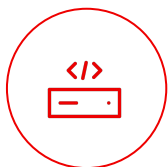
Supported and certified **content you can trust.**

170+

Certified and Validated
Content Collections

70+

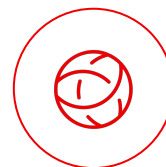
Certified technology
partners



Infrastructure



Cloud



Network



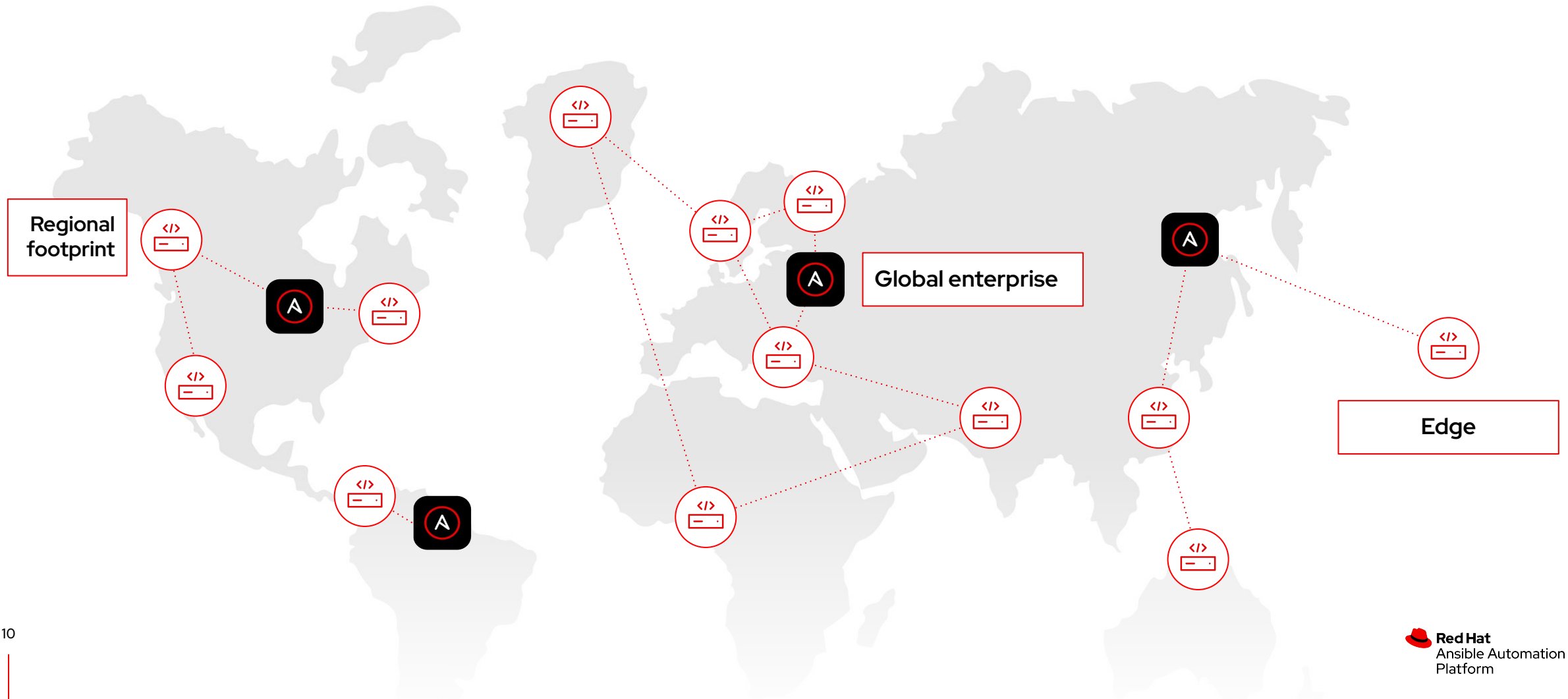
Security



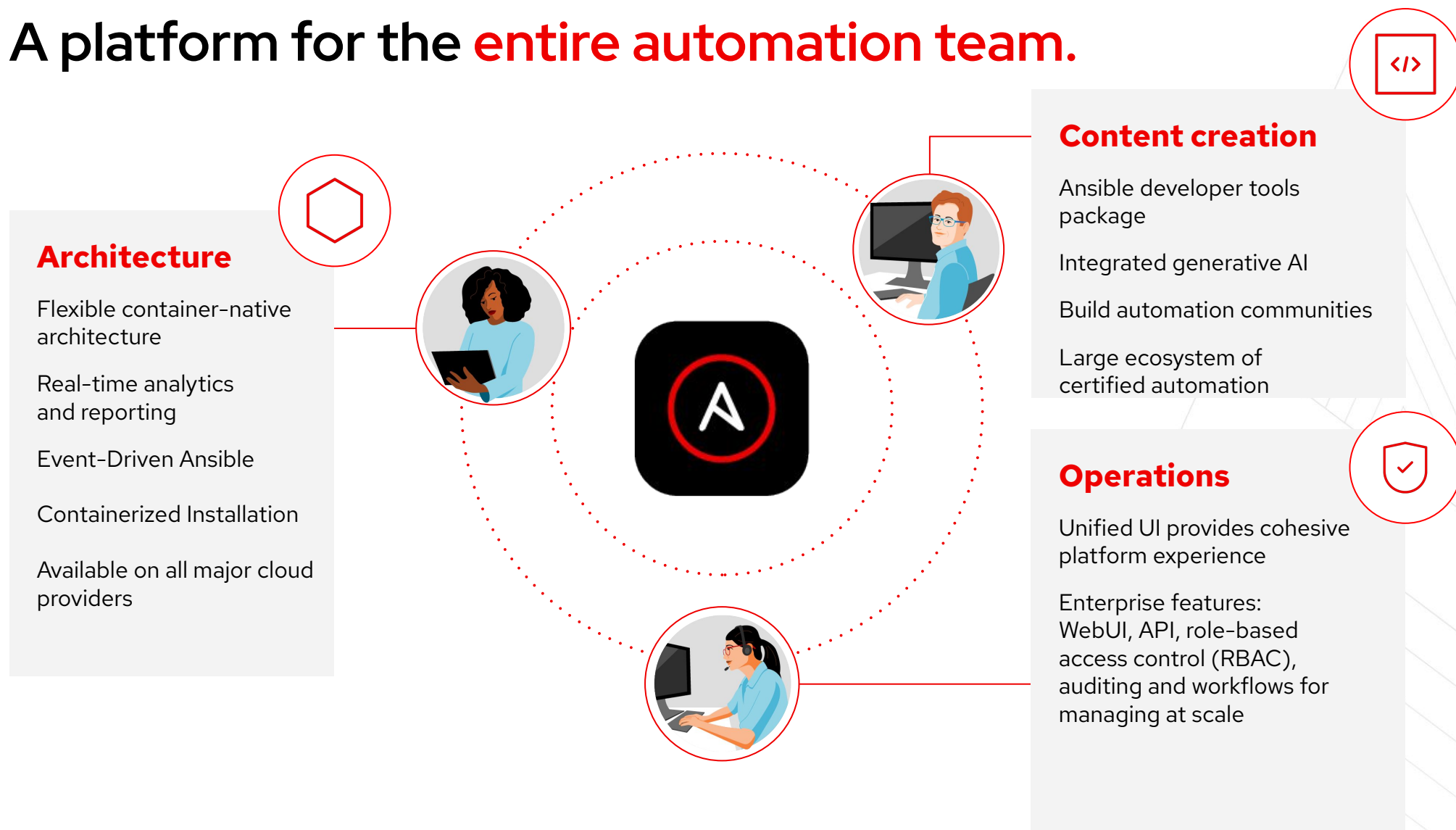
Edge



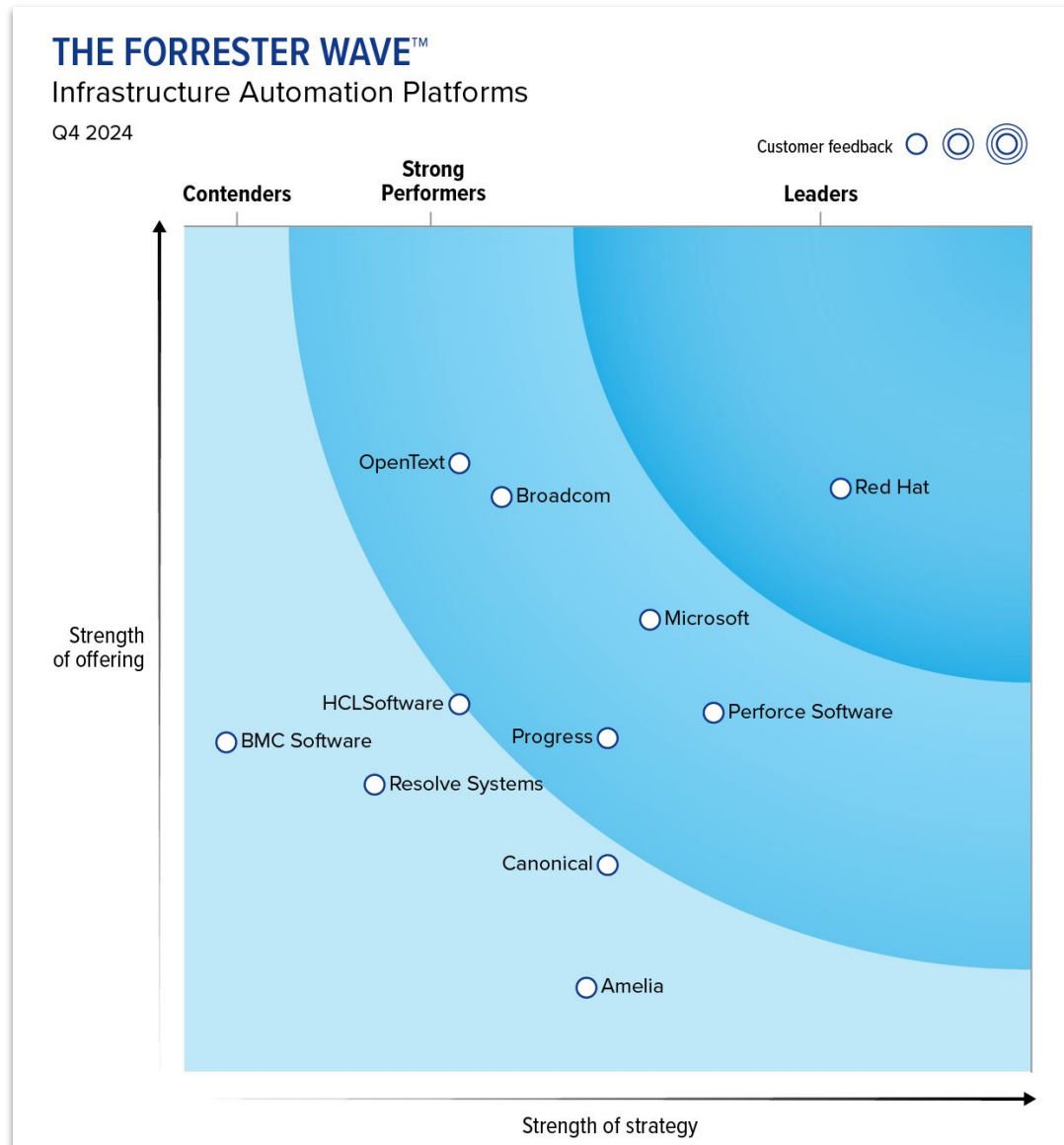
The flexibility to scale, **wherever that may be.**



A platform for the **entire automation team.**



Red Hat is *the leader* in the 2024 Forrester Wave™: Infrastructure Automation Platforms



Key takeaways from vendor profile

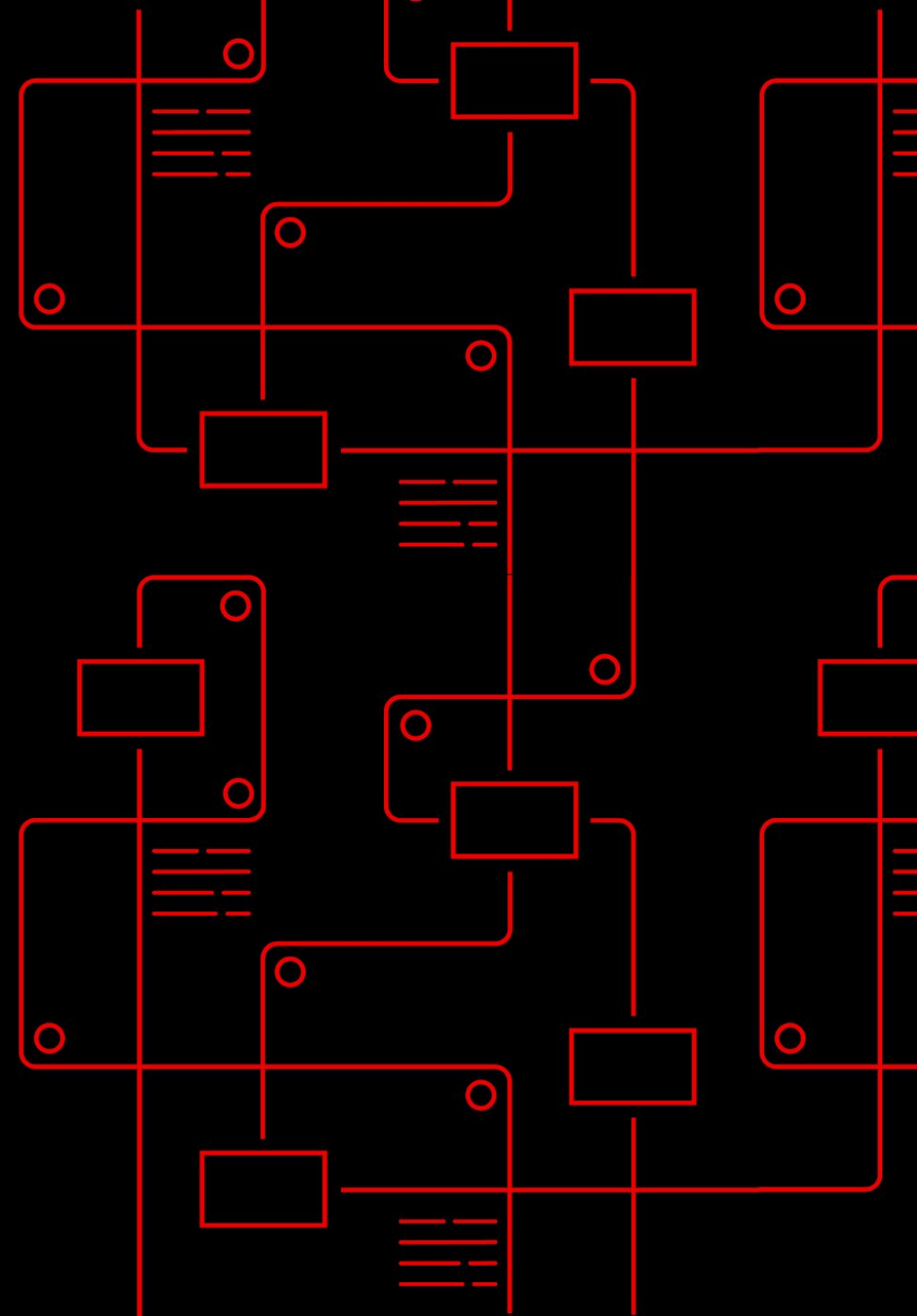
- > "The Ansible Automation Platform is a comprehensive solution for various use cases, expanding into event-driven automation to reduce human intervention and reduce the burden of writing code for each scenario."
- > "Red Hat's strategy for a collective and inclusive user community is spurring growth. Its vision and partner ecosystem are both strengths...supported by the community, Red Hat Ansible addresses numerous automation use cases."
- > "Red Hat offers a good all-around automation tool for organizations that aim to enable users to develop mature automation and integrate technology silos."

Source: Forrester Research, "The Forrester Wave™: Infrastructure Automation Platforms, Q4 2024," 2024.

Forrester does not endorse any company, product, brand, or service included in its research publications and does not advise any person to select the products or services of any company or brand based on the ratings included in such publications. Information is based on the best available resources. Opinions reflect judgment at the time and are subject to change. For more information, read about Forrester's objectivity [here](#).

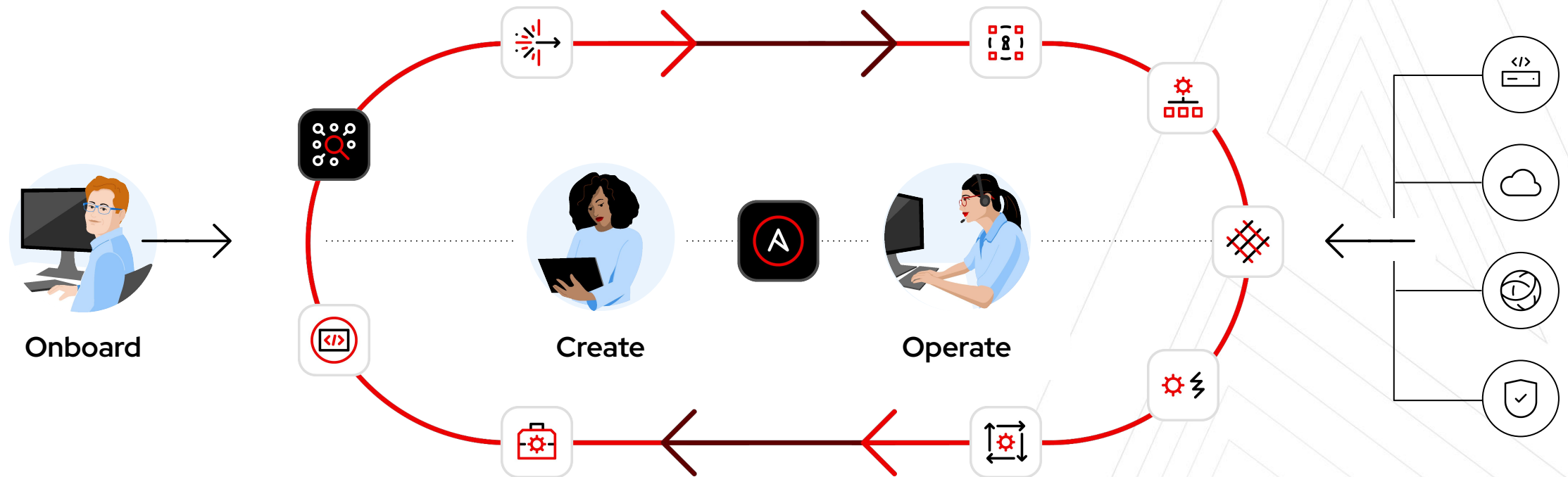
Section 1

The Ansible Basics



Automation Lifecycle: **Development to Management.**

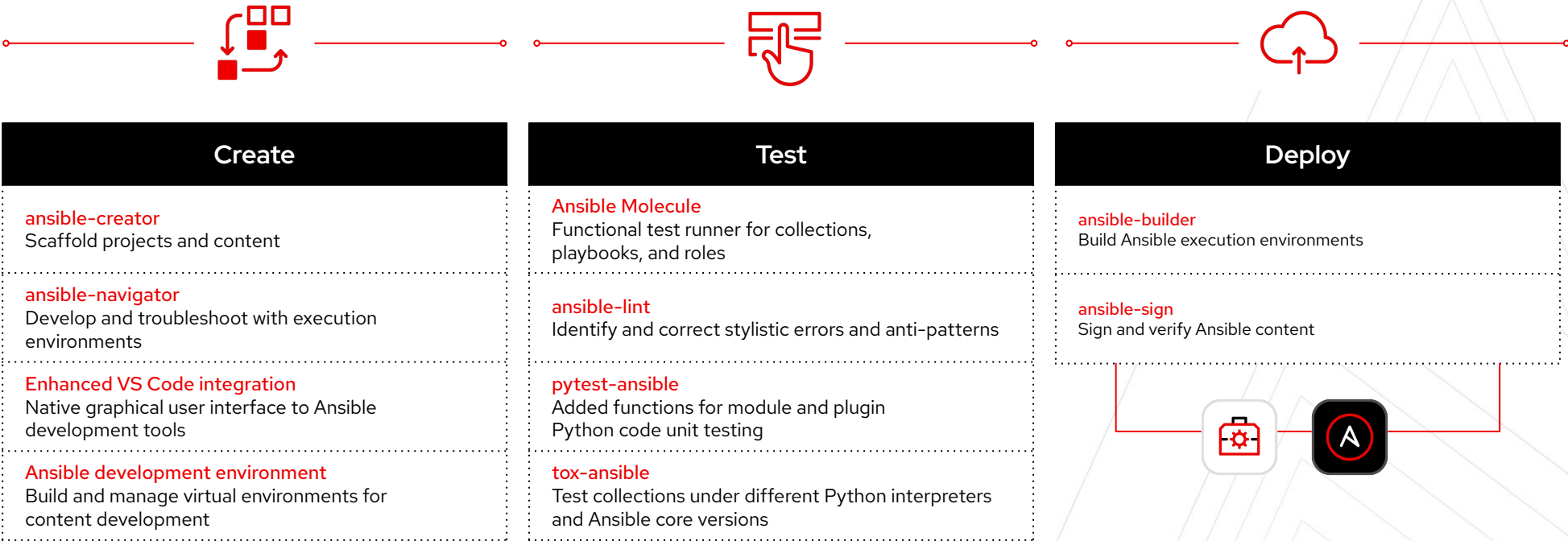
Configuration as Code · Ansible plug-ins for Red Hat Developer Hub · Ansible Lightspeed · Automation hub · Automation Platform UI · Automation mesh



Ansible development workspaces · Ansible development tools · Event-Driven Ansible · Platform Operators

Ansible development tools. **Streamlining creation**

Supported, enterprise-grade components to enable creating, testing, and deployment of Ansible



Ansible playbooks

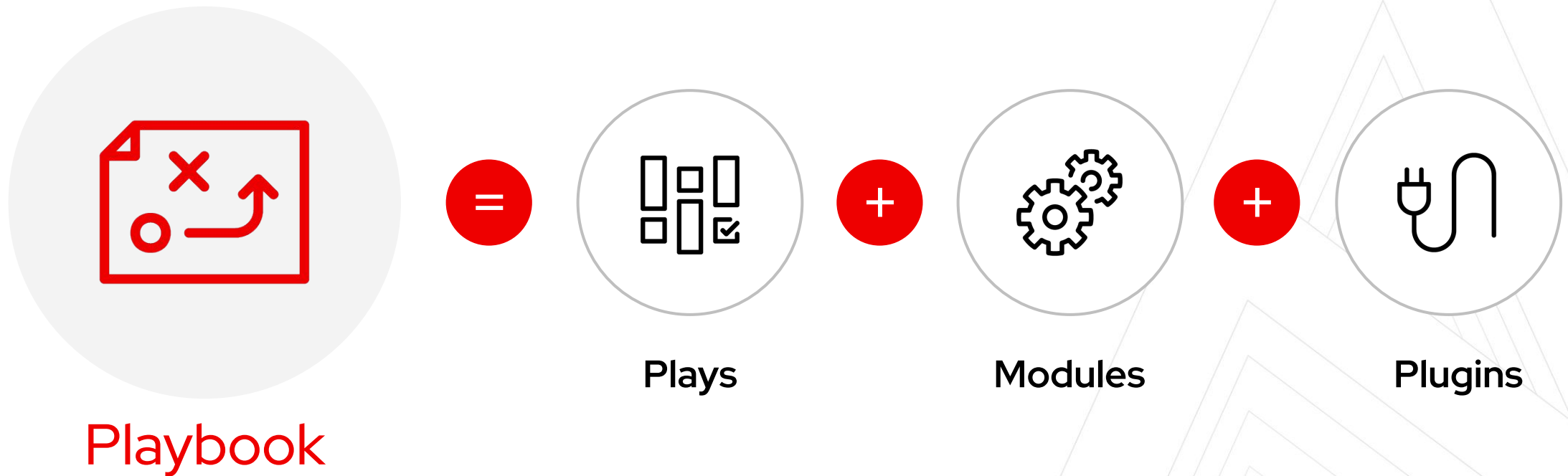
```
---
- name: Install and start apache
  hosts: web
  become: true

  tasks:
    - name: Ensure the httpd package is installed
      ansible.builtin.package:
        name: httpd
        state: present

    - name: Create the index.html file
      ansible.builtin.template:
        src: files/index.html
        dest: /var/www/html/

    - name: Start the httpd service if needed
      ansible.builtin.service:
        name: httpd
        state: started
```

What makes up an Ansible playbook?





Ansible plays. **What am I automating?**

What are they?

- ▶ Top level specification for a group of tasks
- ▶ Will tell that play which hosts it will execute on and control behavior such as fact gathering or privilege level

Building blocks for playbooks

- ▶ Multiple plays can exist within an Ansible playbook

```
---  
- name: Ensure the httpd package is installed  
  hosts: web  
  become: true
```



Ansible modules. The “tools in the toolkit”.

What are they?

- ▶ Parameterized components with internal logic, representing a single step to be done
- ▶ The modules “do” things in Ansible

Language

- ▶ Usually created in Python, or Powershell for Windows setups, but can be developed in any language

```
- name: Create the index.html file
  ansible.builtin.template:
    src: files/index.html
    dest: /var/www/html/
```



Ansible plugins. The “extra bits”.

What are they?

- ▶ Plugins are pieces of code that augment Ansible’s core functionality
- ▶ Ansible uses a plugin architecture to enable a rich, flexible, and expandable feature set

```
---
- name: Example Playbook Using json_query Filter
  hosts: localhost
  gather_facts: no

  vars:
    complex_data:
      users:
        - name: "Alice"
          age: 25
        - name: "Bob"
          age: 30
        - name: "Charlie"
          age: 35

  tasks:
    - name: Extract names of all users
      ansible.builtin.debug:
        msg: "{{ complex_data | community.general.json_query('users[*].name') }}"
```

Ansible Inventory. The systems that a playbook runs against.



What are they?

- List of systems in your infrastructure that automation is executed against

```
[web]
webserver1.example.com
webserver2.example.com

[db]
dbserver1.example.com

[switches]
leaf01.internal.com
leaf02.internal.com
```




Ansible Roles. Reusable automation actions.

What are they?

- ▶ Group tasks and variables of your automation in a reusable structure
- ▶ Write roles once, and share them with others who have similar challenges in front of them

```
---  
- name: Install and start apache  
  hosts: web  
  roles:  
    - common  
    - webservers
```



```
roles
```

```
  nginx
```

```
    defaults
```

```
    files
```

```
    ...
```

```
    tasks
```

```
    templates
```

```
    ...
```

```
  nginx_app_protect
```

```
  nginx_config
```

deploy-nginx.yml

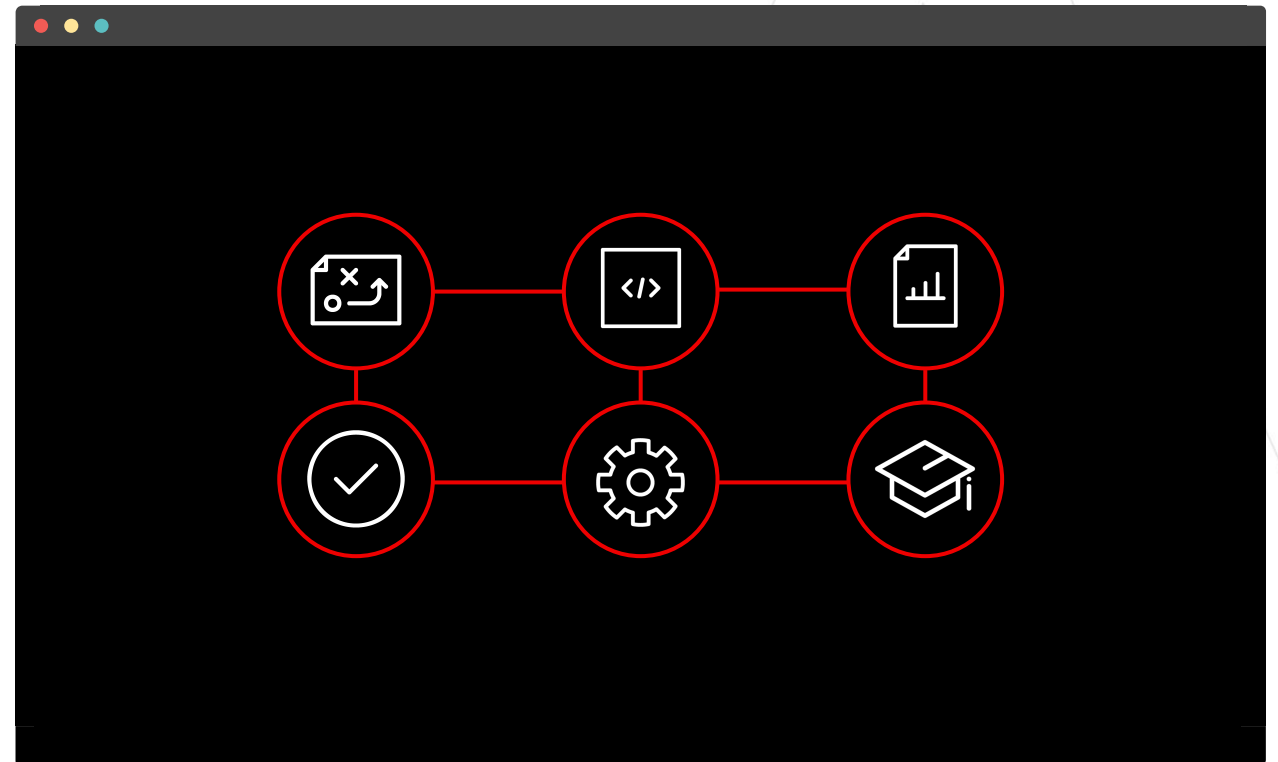
```
---
- name: Install NGINX Plus
  hosts: all
  tasks:
    - name: Install NGINX App Protect
      ansible.builtin.include_role:
        name: nginx_app_protect
      vars:
        nginx_app_protect_setup_license: false
        nginx_app_protect_remove_license: false
        nginx_app_protect_install_signatures: false
```

Content Collections.

Simplified, consistent content delivery.

What are they?

- ▶ Contains automation content, including modules, multiple roles ,and playbooks
- ▶ Portable, reusable, and versioned enabling better collaboration





nginx_core

- ├── galaxy.yml
- ├── meta
- ├── playbooks
 - ├── deploy-nginx.yml
 - └── ...
- ├── plugins
- ├── README.md
- ├── roles
 - ├── nginx
 - ├── defaults
 - ├── files
 - └── ...
 - ├── tasks
 - ├── templates
 - └── ...
 - └── nginx_app_protect
 - └── nginx_config

deploy-nginx.yml

```
---
- name: Install NGINX Plus
  hosts: all
  tasks:
    - name: Install NGINX
      ansible.builtin.include_role:
        name: nginxinc.nginx
      vars:
        nginx_type: plus

    - name: Install NGINX App Protect
      ansible.builtin.include_role:
        name: nginxinc.nginx_app_protect
      vars:
        nginx_app_protect_setup_license: false
        nginx_app_protect_remove_license: false
        nginx_app_protect_install_signatures: false
```

Exercise 1.1

Topics Covered:

- Ansible Inventories
- Ansible Modules
- Ansible Playbooks Basics





Ansible Inventory. The systems that a playbook runs against.

What are they?

- ▶ List of systems in your infrastructure that automation is executed against

How do they work?

- ▶ Defines the systems that Ansible manages and targets for automation.
- ▶ Organizes hosts into groups (e.g., web servers, databases) for better management.
- ▶ Group variables apply settings across multiple systems efficiently.
- ▶ Host-specific variables allow detailed customization for individual systems.

```
[web]
webserver1.example.com
webserver2.example.com

[db]
dbserver1.example.com

[switches]
leaf01.internal.com
leaf02.internal.com
```

Ansible Inventory. The systems that a playbook runs against.



The Basics

- ▶ An example of a static Ansible inventory including systems with IP addresses as well as fully qualified domain name (FQDN)

```
[myservers]
10.42.0.2
10.42.0.6
10.42.0.7
10.42.0.8
10.42.0.100
host.example.com
```

Ansible Inventory

```
[app1srv]
appserver01 ansible_host=10.42.0.2
appserver02 ansible_host=10.42.0.3

[web]
node-[1:30]

[web:vars]
apache_listen_port=8080
apache_root_path=/var/www/mywebdocs/

[all:vars]
ansible_user=kev
ansible_ssh_private_key_file=/home/kev/.ssh/id_rsa
```


Ansible Inventory

```
[app1srv]
appserver01 ansible_host=10.42.0.2
appserver02 ansible_host=10.42.0.3

[web]
node-[1:30]

[web:vars]
apache_listen_port=8080
apache_root_path=/var/www/mywebdocs/

[all:vars]
ansible_user=kev
ansible_ssh_private_key_file=/home/kev/.ssh/id_rsa
```

Ansible playbook

A play

```
---
- name: Install and start apache
  hosts: web
  become: true

  tasks:
    - name: Ensure the httpd package is installed
      ansible.builtin.package:
        name: httpd
        state: present

    - name: Create the index.html file
      ansible.builtin.template:
        src: files/index.html
        dest: /var/www/html/

    - name: Start the httpd service if needed
      ansible.builtin.service:
        name: httpd
        state: started
```

Ansible playbook

A task

```
---  
- name: Install and start apache  
  hosts: web  
  become: true  
  
  tasks:  
    - name: Ensure the httpd package is installed  
      ansible.builtin.package:  
        name: httpd  
        state: present  
  
    - name: Create the index.html file  
      ansible.builtin.template:  
        src: files/index.html  
        dest: /var/www/html/  
  
    - name: Start the httpd service if needed  
      ansible.builtin.service:  
        name: httpd  
        state: started
```

Ansible playbook

A module

```
---
- name: Install and start apache
  hosts: web
  become: true

  tasks:
    - name: Ensure the httpd package is installed
      ansible.builtin.package:
        name: httpd
        state: present

    - name: Create the index.html file
      ansible.builtin.template:
        src: files/index.html
        dest: /var/www/html/

    - name: Start the httpd service if needed
      ansible.builtin.service:
        name: httpd
        state: started
```

Running an Ansible playbook. The most important colors of Ansible.

A task executed as expected, no change was made.

A task executed as expected, making a change.

A task failed to execute successfully.

Running an Ansible Playbook

```
[user@ansible] $ ansible-playbook apache.yml

PLAY [webservers] *****

TASK [Gathering Facts] *****
ok: [web2]
ok: [web1]
ok: [web3]

TASK [Ensure httpd package is present] *****
changed: [web2]
changed: [web1]
changed: [web3]

TASK [Ensure latest index.html file is present] *****
changed: [web2]
changed: [web1]
changed: [web3]

TASK [Restart httpd] *****
changed: [web2]
changed: [web1]
changed: [web3]

PLAY RECAP *****
web2      : ok=1    changed=3 unreachable=0 failed=0
web1      : ok=1    changed=3 unreachable=0 failed=0
web3      : ok=1    changed=3 unreachable=0 failed=0
```

Lab Time

Complete Exercise 1.1



Exercise 1.2

Topics Covered:

- Working with Ansible Variables
- Working with Ansible Facts



Ansible Variable Playbook

```
---
- name: variable playbook test
  hosts: localhost

  vars:
    var_one: awesome
    var_two: ansible is
    var_three: "{{ var_two }} {{ var_one }}"

  tasks:
    - name: print out var_three
      ansible.builtin.debug:
        msg: "{{ var_three }}"
```

Ansible Variable Playbook

```
---
- name: variable playbook test
  hosts: localhost

  vars:
    var_one: awesome
    var_two: ansible is
    var_three: "{{ var_two }} {{ var_one }}"

  tasks:
    - name: print out var_three
      ansible.builtin.debug:
        msg: "{{ var_three }}"
```

ansible is awesome

Ansible Facts

Facts

- ▶ Just like variables, really...
- ▶ ... but: coming from the host itself!
- ▶ Check them out with the setup module

```
tasks:
  - name: Collect all facts of host
    ansible.builtin.setup:
      gather_subset:
        - 'all'
```

Ansible Facts Playbook

```
- name: Ansible Facts playbook
  hosts: localhost
  gather_facts: true

  tasks:
    - name: Define custom fact
      ansible.builtin.set_fact:
        server_role: "frontend"

    - name: Use custom and built-in fact
      ansible.builtin.debug:
        msg: 'The {{ ansible_hostname }} is assigned the role: {{ server_role }}'
```

```
$ ansible-playbook facts_playbook.yml
```

Lab Time

Complete Exercise 1.2



Exercise 1.3

Topics Covered:

- Conditionals
- Handlers
- Loops



Conditionals

What are they?

- ▶ Conditionals allow tasks to run **only if certain conditions are met.**
- ▶ Enable **dynamic automation** by checking the values of variables & making decisions at runtime.

```
vars:
  my_mood: happy

tasks:
- name: task, based on my_mood var
  ansible.builtin.debug:
    msg: "Yay! I am {{ my_mood }}!"
  when: my_mood == "happy"
```

Ansible Conditionals

```
---
- name: variable playbook test
  hosts: localhost

  vars:
    my_mood: happy

  tasks:
    - name: task, based on my_mood var
      ansible.builtin.debug:
        msg: "Yay! I am {{ my_mood }}"
      when: my_mood == "happy"
```

Alternatively

```
- name: task, based on my_mood var
  ansible.builtin.debug:
    msg: "Ask at your own risk. I'm {{ my_mood }}"
  when: my_mood == "grumpy"
```


Ansible Conditionals with Facts

```
---
- name: variable playbook test
  hosts: localhost

  tasks:
    - name: Install httpd
      ansible.builtin.package:
        name: httpd
        state: latest
      when: ansible_distribution == 'RedHat'

    - name: Install apache
      ansible.builtin.package:
        name: apache2
        state: latest
      when: ansible_distribution == 'Debian' or
            ansible_distribution == 'Ubuntu'
```

Ansible Conditionals using Previous Task State

```
---
- name: variable playbook test
  hosts: localhost

  tasks:
    - name: Ensure httpd package is present
      ansible.builtin.package:
        name: httpd
        state: latest
        register: httpd_results

    - name: Restart httpd
      ansible.builtin.service:
        name: httpd
        state: restarted
        when: httpd_results.changed
```

Ansible Handler Tasks

```
---
- name: variable playbook test
  hosts: localhost

  tasks:
    - name: Ensure httpd package is present
      ansible.builtin.package:
        name: httpd
        state: latest
      notify: restart_httpd

  handlers:
    - name: restart_httpd
      ansible.builtin.service:
        name: httpd
        state: restarted
```

Ansible Handler Tasks

tasks:

- name: Ensure httpd package is present
ansible.builtin.package:
 name: httpd
 state: latest
 notify: restart httpd
- name: Standardized index.html file
ansible.builtin.copy:
 content: "This is my index.html file for {{ ansible_host }}"
 dest: /var/www/html/index.html
 notify: restart httpd

If **either** task
notifies a
changed result,
the handler will be
notified **ONCE**.

```
TASK [Ensure httpd package is present] *****
```

```
ok: [web2] unchanged  
ok: [web1]
```

```
TASK [Standardized index.html file] *****
```

```
changed: [web2] changed  
changed: [web1]
```

```
NOTIFIED: [restart_httpd] *****
```

```
changed: [web2]  
changed: [web1]
```

handler runs once

Ansible Handler Tasks

tasks:

- name: Ensure httpd package is present
ansible.builtin.package:
 name: httpd
 state: latest
 notify: restart httpd
- name: Standardized index.html file
ansible.builtin.copy:
 content: "This is my index.html file for {{ ansible_host }}"
 dest: /var/www/html/index.html
 notify: restart httpd

If **both** of these tasks notifies of a **changed** result, the handler will be notified **ONCE**.

```
TASK [Ensure httpd package is present] *****
changed: [web2]
changed: [web1] changed
```

```
TASK [Standardized index.html file] *****
changed: [web2]
changed: [web1] changed
```

```
NOTIFIED: [restart_httpd] *****
changed: [web2]
changed: [web1]
```

handler runs once

Ansible Handler Tasks

tasks:

- name: Ensure httpd package is present
 ansible.builtin.package:
 name: httpd
 state: latest
 notify: restart httpd
- name: Standardized index.html file
 ansible.builtin.copy:
 content: "This is my index.html file for {{ ansible_host }}"
 dest: /var/www/html/index.html
 notify: restart httpd

If **neither** task
notifies a
changed result,
the handler
does not run.

```
TASK [Ensure httpd package is present] *****
```

```
ok: [web2]           unchanged  
ok: [web1]
```

```
TASK [Standardized index.html file] *****
```

```
ok: [web2]           unchanged  
ok: [web1]
```

```
PLAY RECAP *****
```

```
web2      : ok=2   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0  
web1      : ok=2   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Ansible Variables and Loops

```
---
- name: Ensure users
  hosts: node1
  become: true

  tasks:
    - name: Ensure user is present
      ansible.builtin.user:
        name: dev_user
        state: present

    - name: Ensure user is present
      ansible.builtin.user:
        name: qa_user
        state: present

    - name: Ensure user is present
      ansible.builtin.user:
        name: prod_user
        state: present
```

Ansible Variables and Loops

```
---
- name: Ensure users
  hosts: node1
  become: true

  tasks:
    - name: Ensure user is present
      ansible.builtin.user:
        name: "{{ item }}"
        state: present
      loop:
        - dev_user
        - qa_user
        - prod_user
```


Lab Time

Complete Exercise 1.3



Exercise 1.4

Topics Covered:

- Templates





Templates

What are they?

- ▶ Templates are files that dynamically generate content by using variables, conditionals, loops and filters.

How do they work?

- ▶ First by reading in the source template file (.j2)
- ▶ Filling in the dynamic values based on the variables or facts
- ▶ Deploy the processed file with the variable values in place

```
---
- name: Ensure apache is installed and started
  hosts: web
  become: true
  vars:
    http_port: 80
    http_docroot: /var/www/mysite.com

  tasks:
    - name: Verify correct config file is present
      ansible.builtin.template:
        src: templates/httpd.conf.j2
        dest: /etc/httpd/conf/httpd.conf
```

Ansible Templates

```
- name: Ensure apache is installed and started
hosts: web
become: true
```

```
vars:
  http_port: 80
  http_docroot: /var/www/mysite.com
```

tasks:

```
- name: Verify correct config file is present
  ansible.builtin.template:
    src: templates/httpd.conf.j2
    dest: /etc/httpd/conf/httpd.conf
```

```
## Excerpt from httpd.conf.j2
```

```
# Change this to Listen on specific IP addresses as shown below to
# prevent Apache from glomming onto all bound IP addresses.
```

```
#
```

```
# Listen 80    ## original line
```

```
Listen {{ http_port }}
```

```
# DocumentRoot: The directory out of which you will serve your
# documents.
```

```
# DocumentRoot "/var/www/html"
```

```
DocumentRoot {{ http_docroot }}
```

Lab Time

Complete Exercise 1.4



Exercise 1.5

Topics Covered:

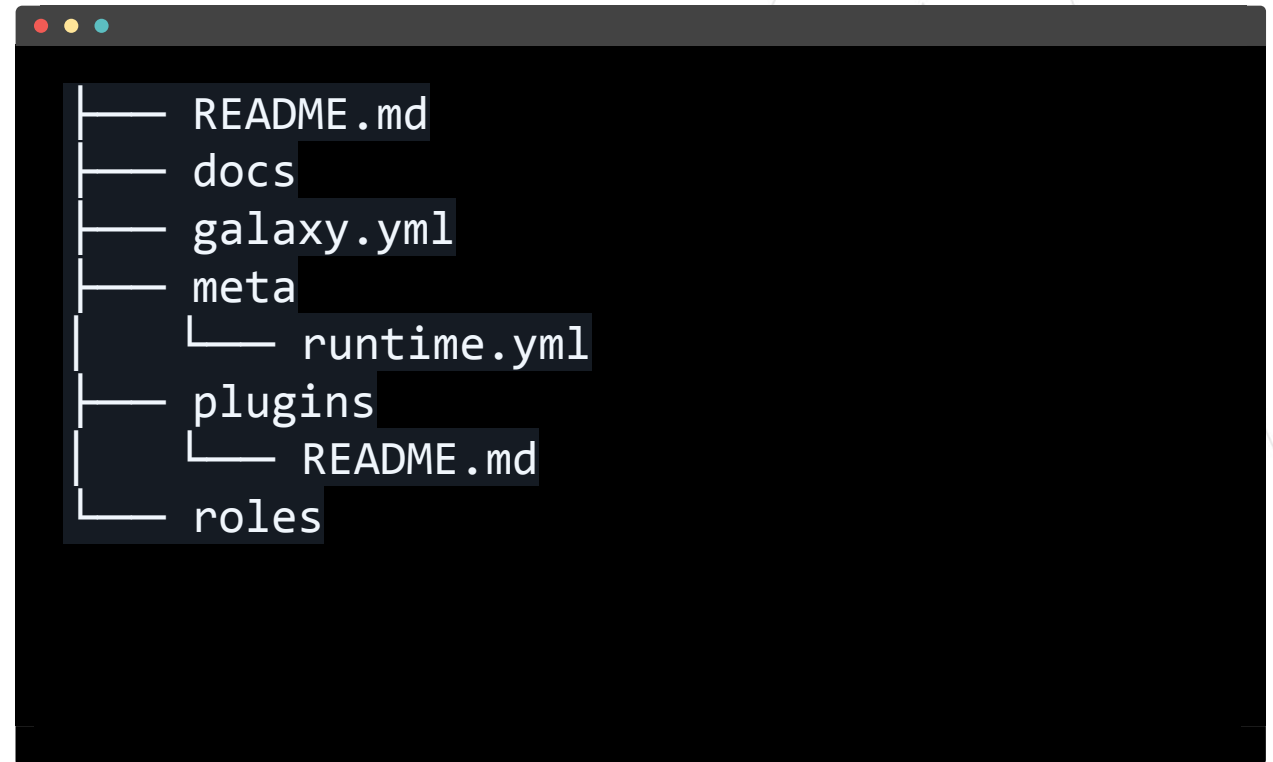
- What are Ansible Collections?
- How do you create an Ansible Collection?
- What and how do I use ansible-galaxy?



Ansible Collections

What are they?

- ▶ A way to **organize**, **distribute**, and **reuse** automation content.
- ▶ Group components like **roles**, **modules**, **plugins** and **playbooks**.
- ▶ Distributed via:
 - Ansible Galaxy
 - Automation Hub
- ▶ Improves **content management** and **collaboration** within teams



Ansible Galaxy

What is Ansible Galaxy?

- ▶ **Ansible Galaxy** is a **community platform** to **discover, share, and download** automation content like **roles** and **collections**.
- ▶ It enables users to **reuse existing content** instead of building everything from scratch, fostering collaboration and efficiency.

```
# Install a collection from Ansible Galaxy
$ ansible-galaxy collection install community.general

# List installed collections

$ ansible-galaxy collection list
```



Lab Time

Complete Exercise 1.5



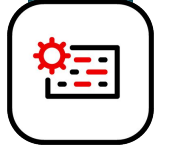
Exercise 1.6

Topics Covered:

- Ansible Navigator
- Execution Environments



Running an Ansible Playbook with `ansible-navigator`



What is it?

It is a command line utility and text-based user interface (TUI) for running, testing and developing Ansible automation content

- ▶ Review EEs
- ▶ Develop collections
- ▶ Develop playbooks
- ▶ Troubleshoot problems

```
# Direct command-line interface method
$ ansible-navigator run playbook.yml \
-i inventory.ini \
-m stdout

# Text-based User Interface method
$ ansible-navigator run playbook.yml -i inventory.ini
```

Ansible content navigator (`ansible-navigator`)

Mapping to previous Ansible commands

ansible command	ansible-navigator command
<code>ansible-config</code>	<code>ansible-navigator config</code>
<code>ansible-doc</code>	<code>ansible-navigator doc</code>
<code>ansible-inventory</code>	<code>ansible-navigator inventory</code>
<code>ansible-playbook</code>	<code>ansible-navigator run</code>

Ansible content navigator (`ansible-navigator`)

Common subcommands

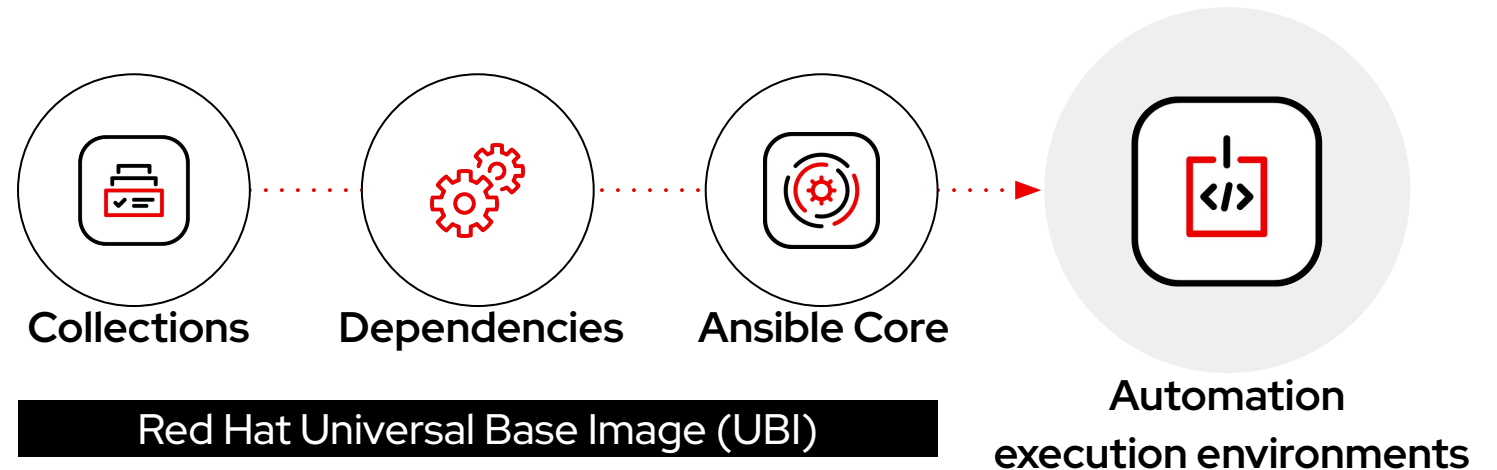
Name	Description	CLI Example	Colon command within TUI
<code>collections</code>	Explore available collections	<code>ansible-navigator collections --help</code>	<code>:collections</code>
<code>config</code>	Explore the current ansible configuration	<code>ansible-navigator config --help</code>	<code>:config</code>
<code>doc</code>	Review documentation for a module or plugin	<code>ansible-navigator doc --help</code>	<code>:doc</code>
<code>images</code>	Explore execution environment images	<code>ansible-navigator images --help</code>	<code>:images</code>
<code>inventory</code>	Explore and inventory	<code>ansible-navigator inventory --help</code>	<code>:inventory</code>
<code>replay</code>	Explore a previous run using a playbook artifact	<code>ansible-navigator replay --help</code>	<code>:replay</code>
<code>run</code>	Run a playbook	<code>ansible-navigator run --help</code>	<code>:run</code>
<code>welcome</code>	Start at the welcome page	<code>ansible-navigator welcome --help</code>	<code>:welcome</code>

Automation execution environments.

Reuse and scale automation content.

What are they?

- ▶ Containerized environments built using an Red Hat Universal Base Image that bundles essential collections, dependencies and Ansible core to ensure consistent and portable automation.
- ▶ Provide a reliable and repeatable way to run your automation consistently throughout your automation lifecycle.



Lab Time

Complete Exercise 1.6



Exercise 1.7

Topics Covered:

- Debugging in Ansible
- Error Handling with Ansible



Debugging in Ansible. Identify and Resolve Issues.



How does it work?

- ▶ Debugging helps identify and resolve issues in playbooks.
- ▶ Ansible offers several methods for debugging, including:
 - Debug module
 - Increased verbosity levels

```
- name: Display Variable Value
ansible.builtin.debug:
  var: apache_service_name

- name: Display Custom Message
ansible.builtin.debug:
  msg: "Apache service name is {{ apache_service_name }}"
```

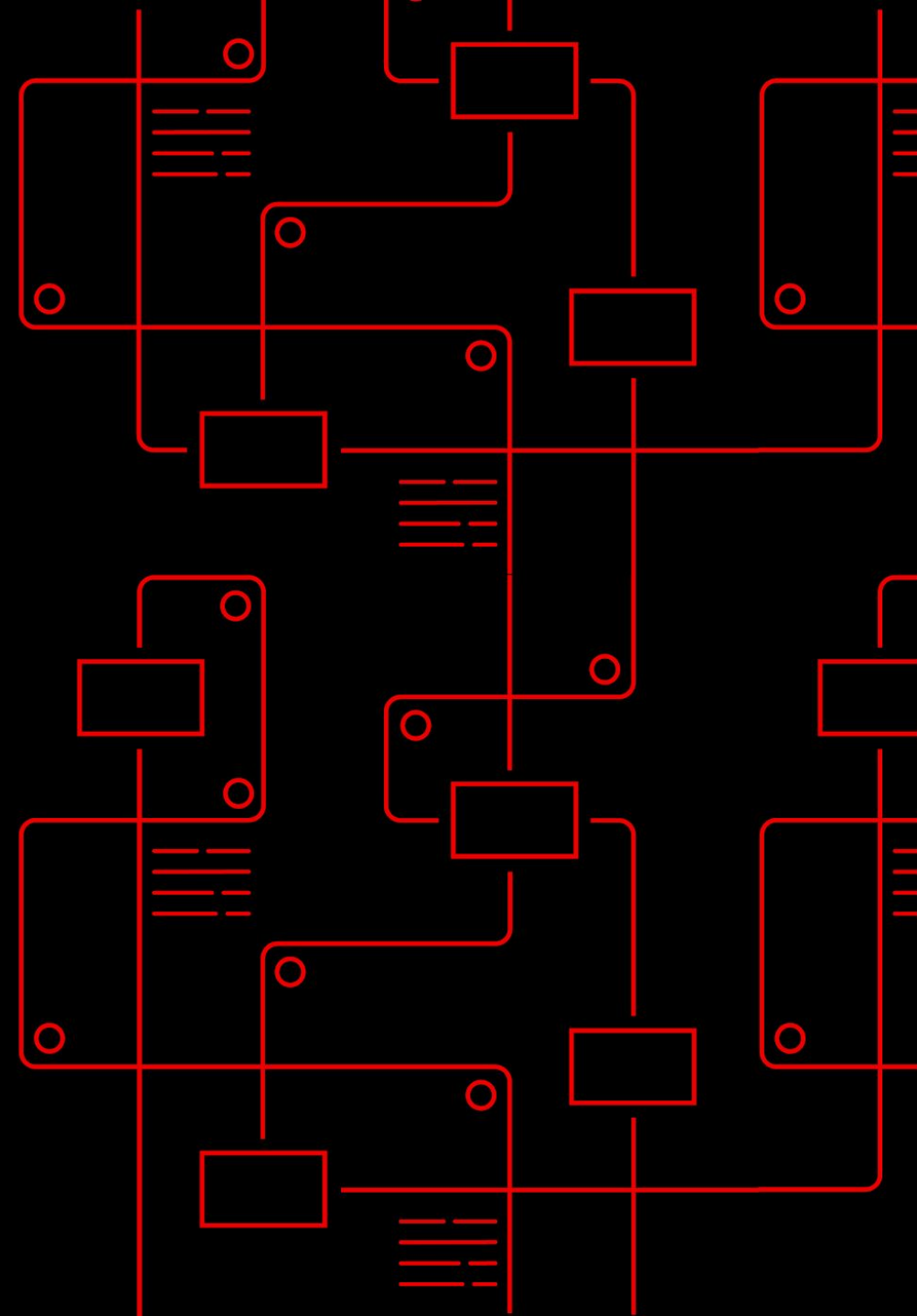
Lab Time

Complete Exercise 1.7



Section 2

Automation Controller



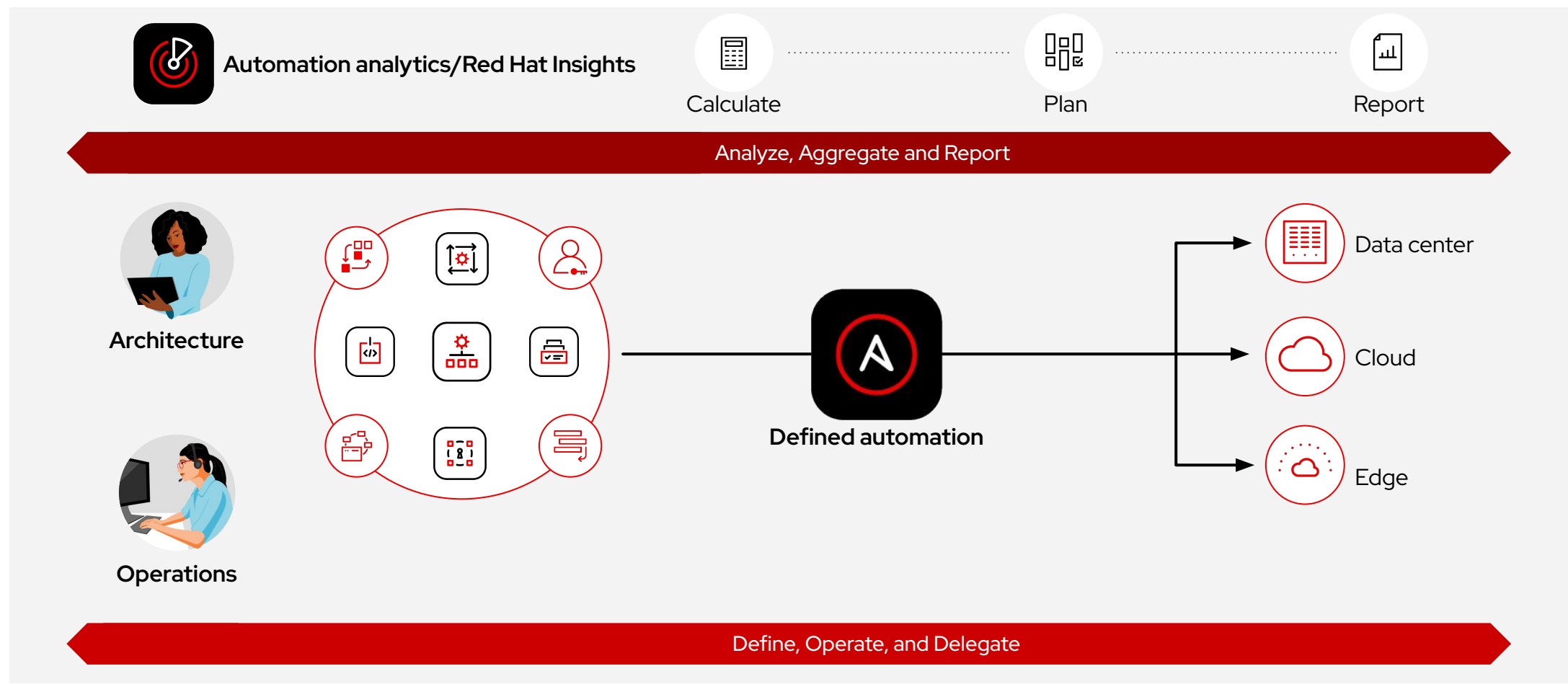
Exercise 2.1

Topics Covered:

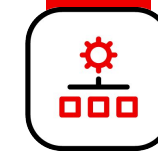
- Intro to Automation Controller



The automation life cycle



Automation controller. Define, operate, and delegate.

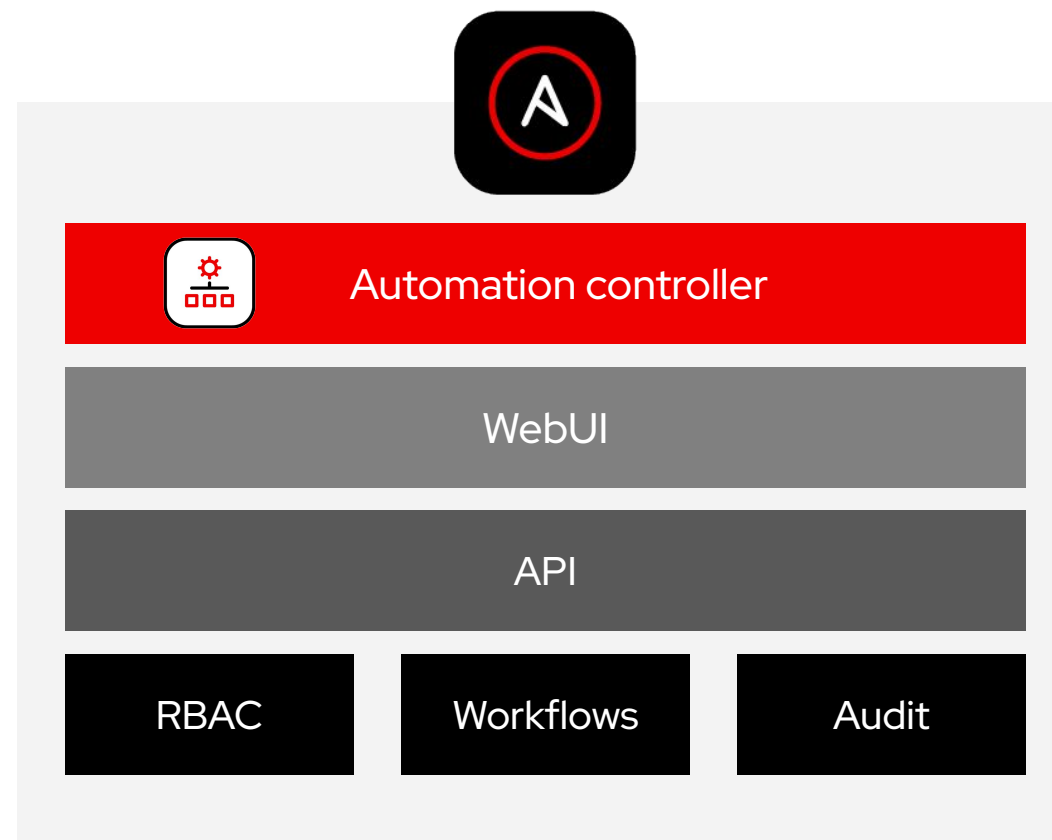


What is it?

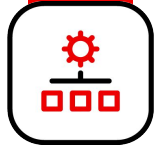
Automation controller is the Ansible Automation Platform control plane which enables users to define, operate, and delegate automation across their enterprise

Automation controller provides:

- ▶ WebUI and API
- ▶ Role-based access control
- ▶ Powerful workflows
- ▶ Centralized logging
- ▶ Credential management
- ▶ Push-button automation



Automation controller. Define, operate, and delegate.



Push button

An intuitive user interface experience makes it easy for novice users to execute playbooks you allow them access to.

RESTful API

With an API first mentality every feature and function of controller can be API driven. Allow seamless integration with other tools like ServiceNow and Infoblox.

RBAC

Allow restricting playbook access to authorized users. One team can use playbooks in check mode (read-only) while others have full administrative abilities.

Centralized logging

All automation activity is securely logged. Who ran it, how they customized it, what it did, where it happened – all securely stored and viewable later, or exported through Automation controllers API.

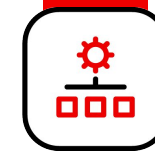
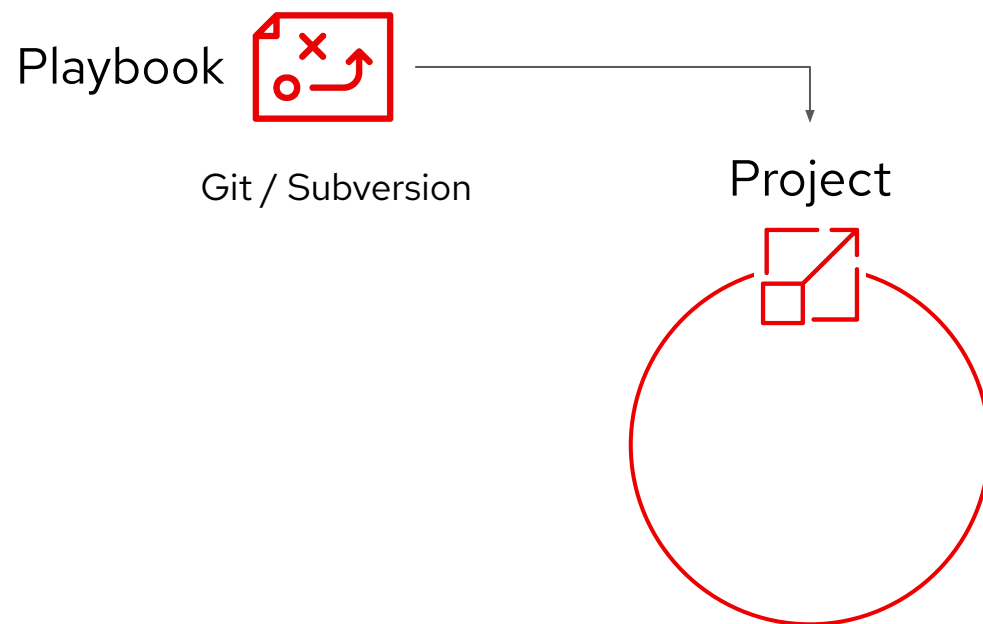
Workflows

Automation controller's multi-playbook workflows chain any number of playbooks, regardless of whether they use different inventories, run as different users, run at once or utilize different credentials.

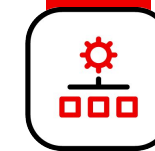
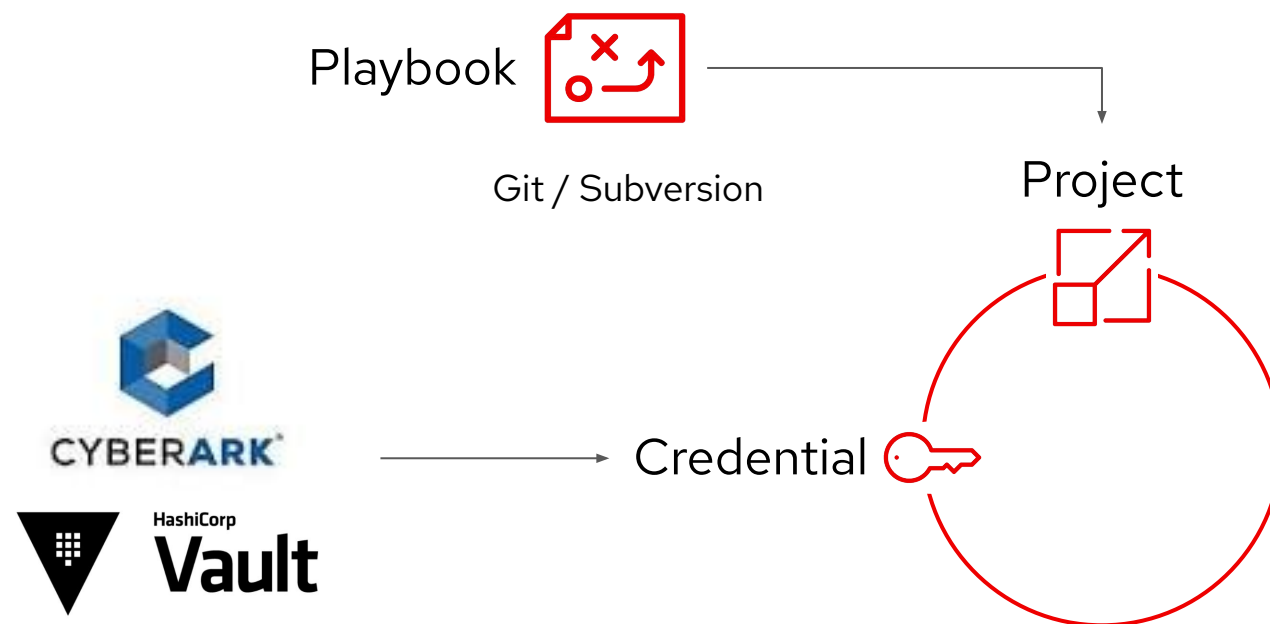
Credential Management

Ability to access and authenticate with external resources, repositories, or target endpoints.

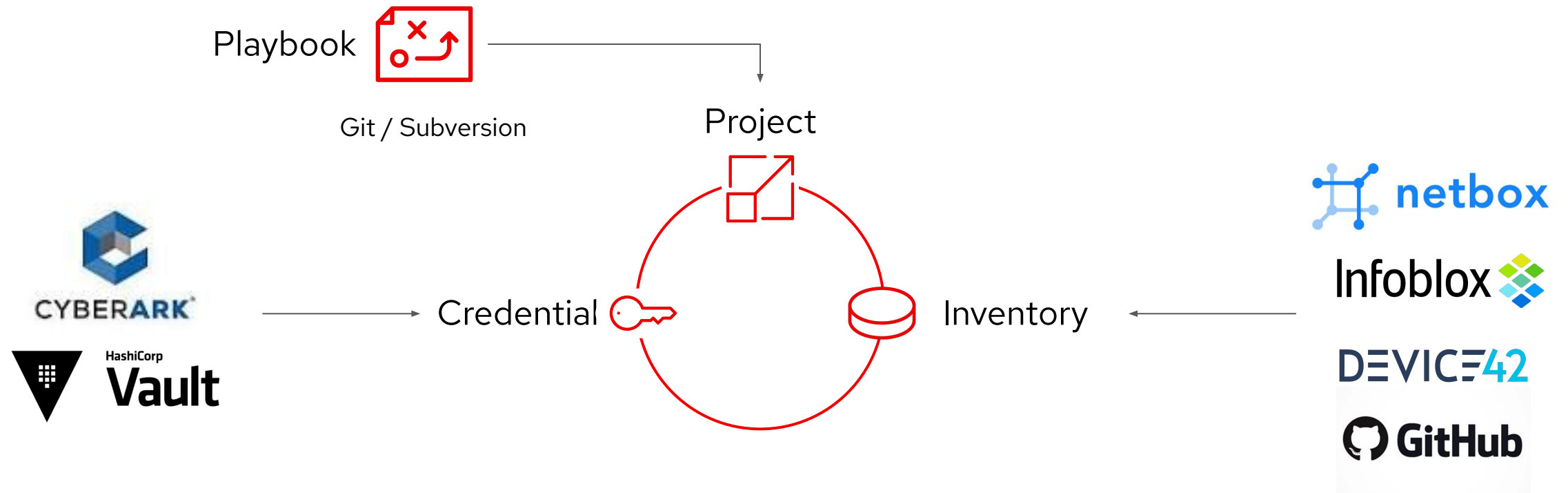
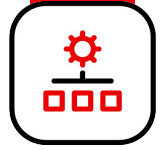
Anatomy of an Automation Job



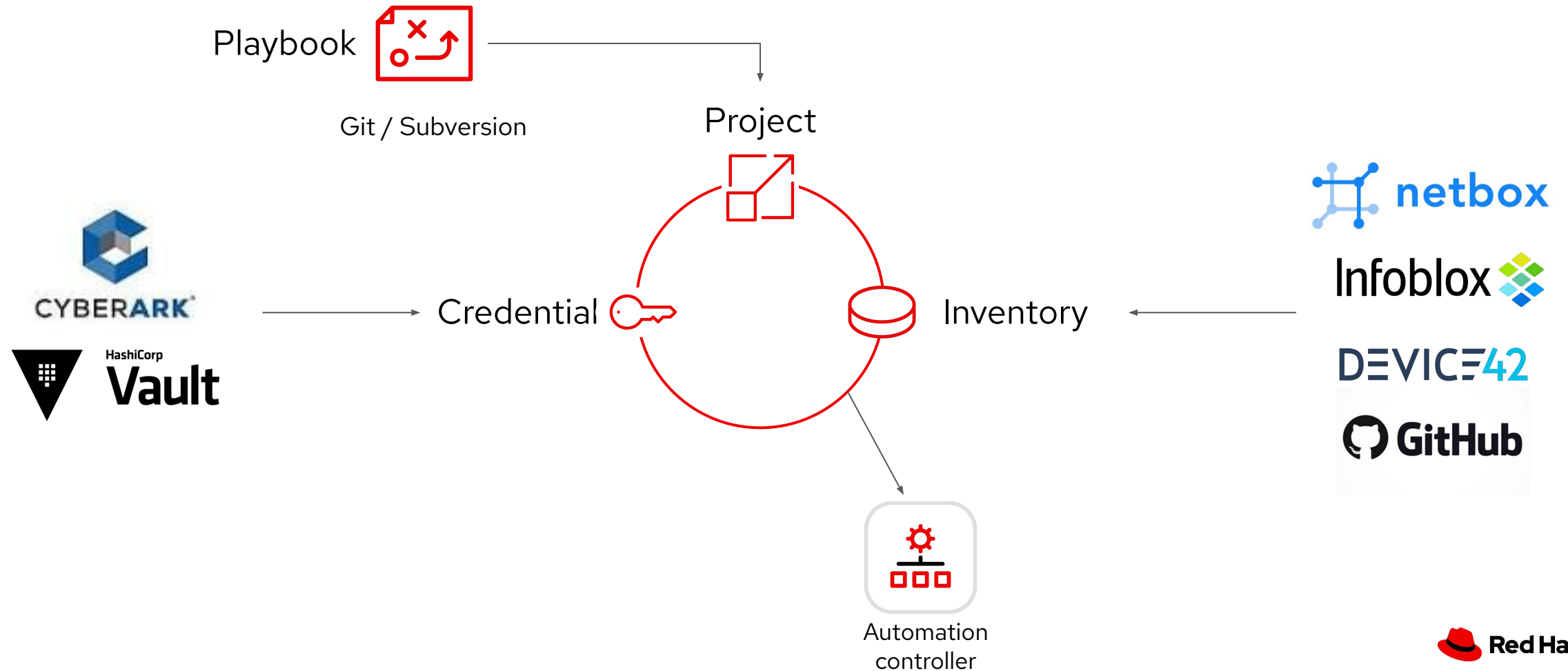
Anatomy of an Automation Job



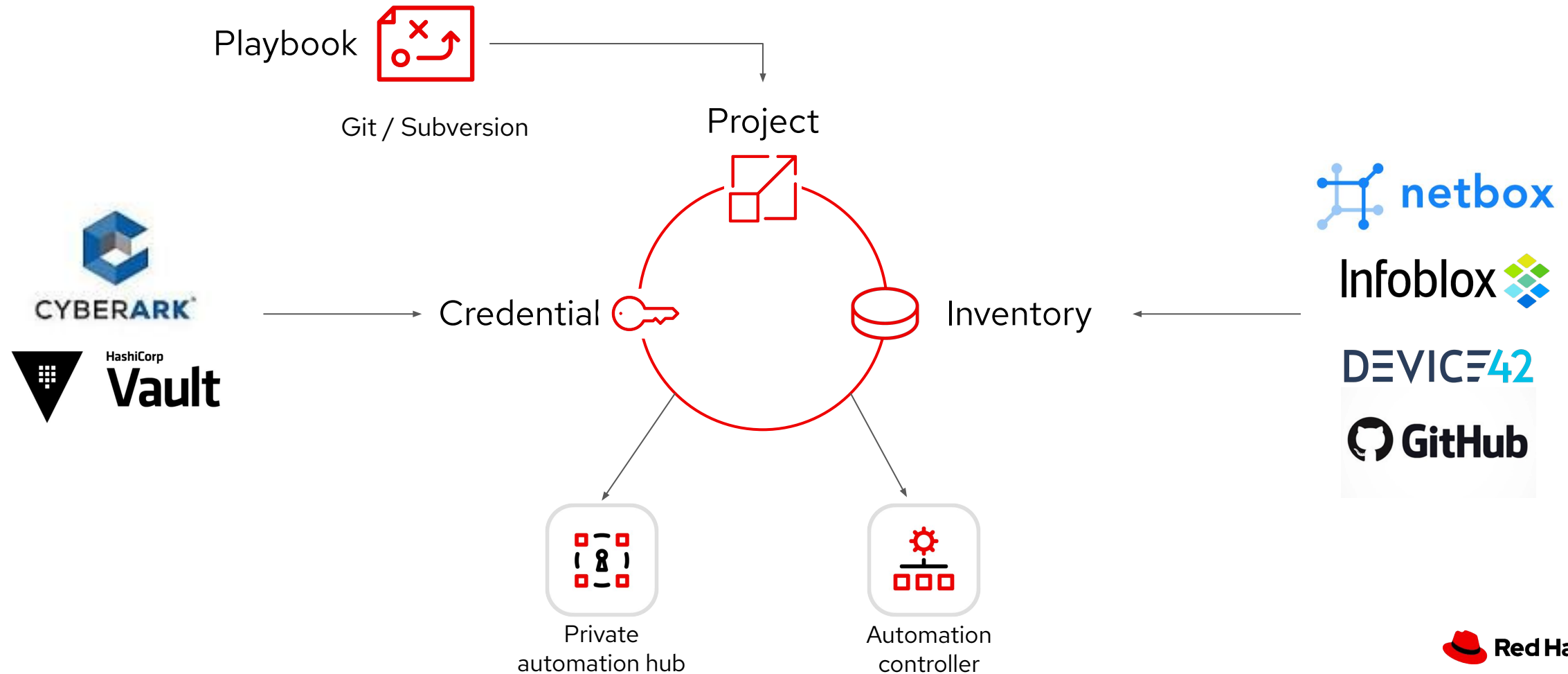
Anatomy of an Automation Job



Anatomy of an Automation Job



Anatomy of an Automation Job



Lab Time

Complete Exercise 2.1



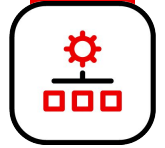
Exercise 2.2

Topics Covered:

- Inventories
- Credentials



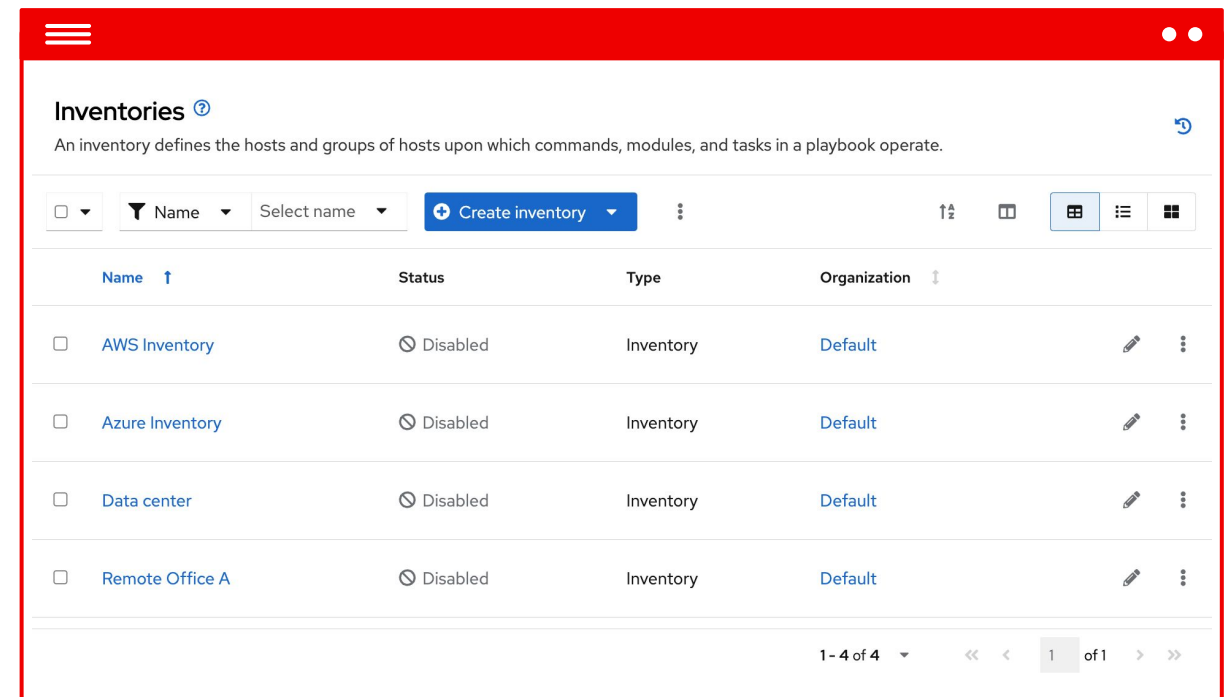
Inventories. What do I want to run my automation on?



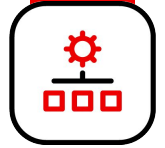
What is it?

Collection of endpoints against which jobs may be launched

- ▶ Multiple inventory sources supported
- ▶ Dynamic endpoint discovery
- ▶ Logically group endpoints by metadata or user-defined filters
- ▶ Granular RBAC permissions

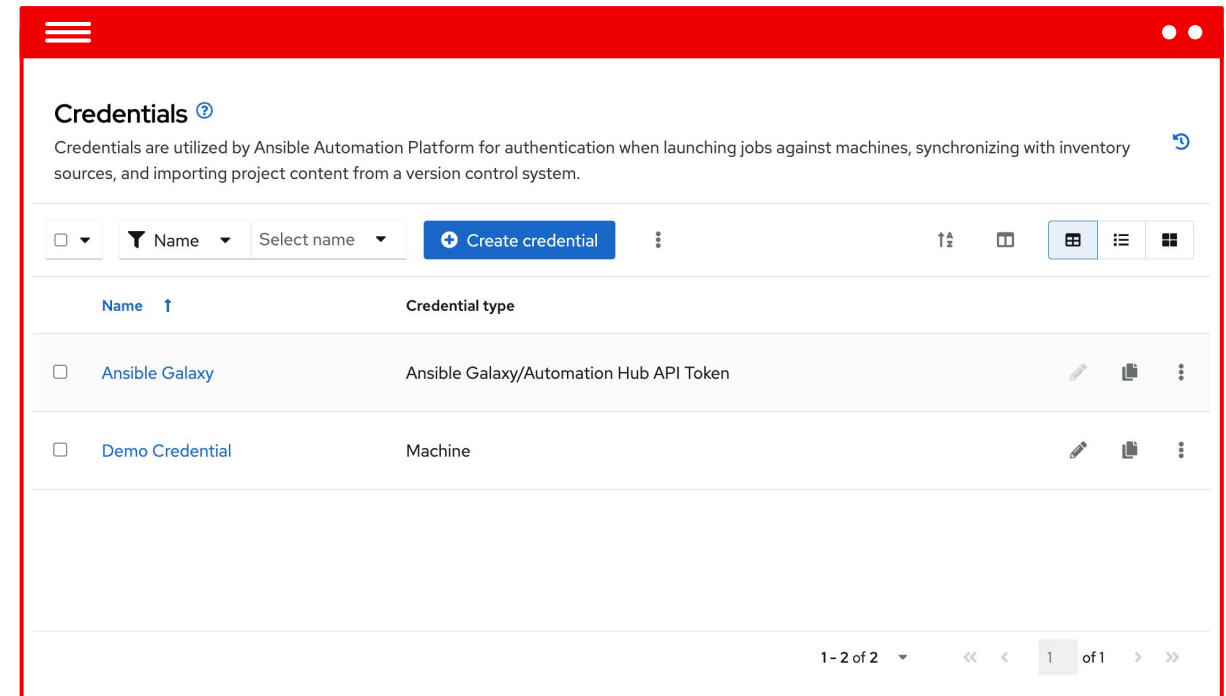


Credentials. **Securing resource and endpoint access.**



What is it?

- ▶ Securely manage credentials needed for automation resources
- ▶ Multiple credential types supported
- ▶ Integrate external secret management systems
- ▶ Create custom credential types and plugins
- ▶ Use RBAC to govern access
- ▶ Actual credential never exposed



Lab Time

Complete Exercise 2.2



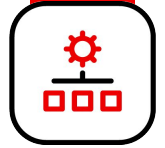
Exercise 2.3

Topics Covered:

- Projects
- Job Templates



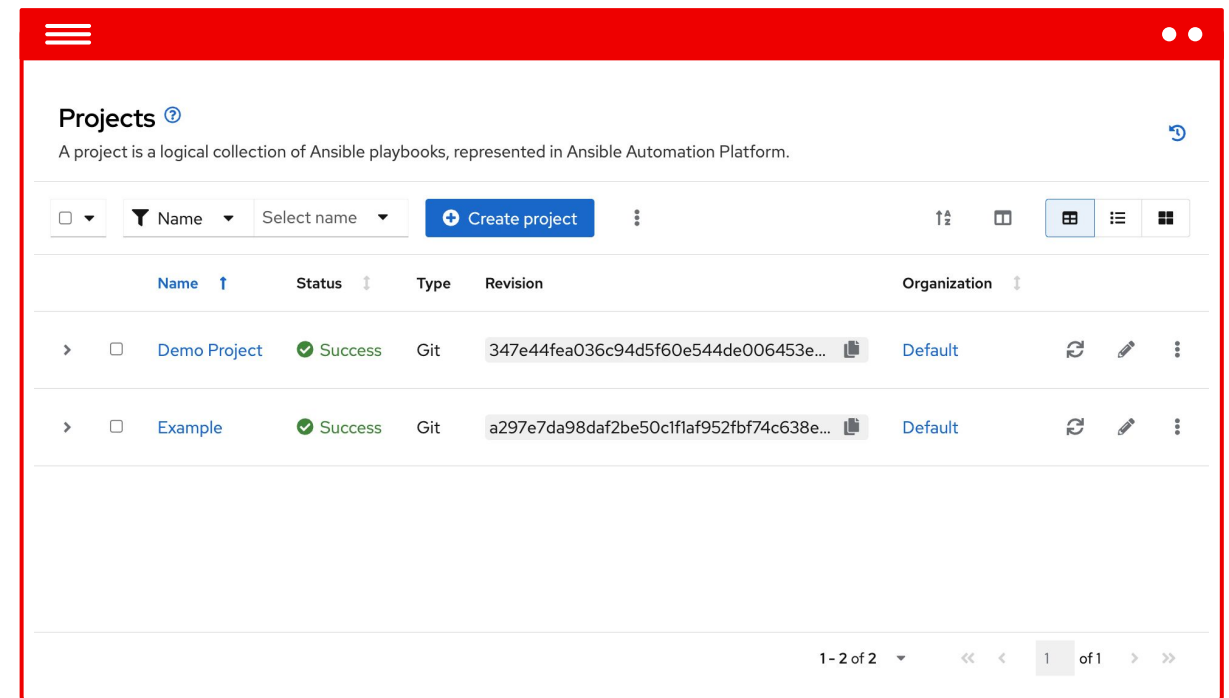
Projects. Adding your automation content to controller.



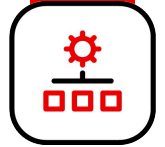
What is it?

Logical collection of your playbooks:

- ▶ Multiple source types supported
- ▶ Source Control Management (SCM) integration and update strategies
- ▶ Role-based access control (RBAC) and schedules



Job Templates. Bringing it all together.

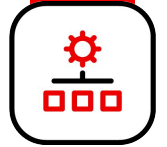


What is it?

- ▶ Define and standardize running automation
- ▶ Reusable and shareable
- ▶ Leverage agile practices, such as GitOps and event-driven automation

The screenshot displays the 'Edit Demo Job Template' interface. It features a red header bar with a menu icon and window controls. The main content area is white and contains a form with various configuration options. The form is organized into sections with expandable/collapsible headers. The 'Name' field is set to 'Demo Job Template'. The 'Description' field is 'Enter description'. The 'Job type' is 'Run'. The 'Inventory' is 'Demo Inventory'. The 'Project' is 'Demo Project'. The 'Playbook' is 'hello_world.yml'. The 'Execution environment' is 'Select execution environment'. The 'Credentials' is 'Demo Credential'. The 'Labels' are 'Select or create labels'. The 'Forks' is '0'. The 'Limit' is 'Enter limit to reduce number of hosts'. The 'Verbosity' is '0'. The 'Job slicing' is '1'. The 'Timeout' is '0'. The 'Show changes' is a toggle switch. The 'Instance groups' are 'Select instance groups'. The 'Job tags' are 'Select or create job tags'. The 'Skip tags' are 'Select or create skip tags'. The 'Extra variables' section is expanded, showing a text area for variables. At the bottom, there are checkboxes for 'Privilege escalation', 'Concurrent jobs', 'Provisioning callback', 'Enable fact storage', 'Enable webhook', and 'Prevent instance group fallback'. A 'Save job template' button and a 'Cancel' link are at the bottom left.

Automation jobs. Executing your defined automation.



What is it?

- ▶ Controller launching an instance of defined automation
- ▶ Relaunch automation jobs
- ▶ Use Job Details to view job outputs
- ▶ Troubleshoot automation execution using filtered views

ID	Name	Status	Type	Duration	Started	Finished
9688	Demo Job Template	Pending	Playbook run			--
9694	Cleanup Job Details	Success	Management job	4s	10/6/2024, 9:00:47 AM	10/6/2024, 9:00:51 AM
9693	Cleanup Expired OAuth 2 Tokens	Success	Management job	4s	10/2/2024, 9:03:10 AM	10/2/2024, 9:03:15 AM
9692	Cleanup Expired Sessions	Success	Management job	3s	10/2/2024, 9:02:47 AM	10/2/2024, 9:02:51 AM
9691	Cleanup Activity Stream	Success	Management job	4s	10/1/2024, 9:00:47 AM	10/1/2024, 9:00:52 AM
9690	Cleanup Job Details	Success	Management job	4s	9/29/2024, 9:00:47 AM	9/29/2024, 9:00:51 AM

Lab Time

Complete Exercise 2.3



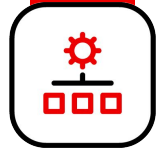
Exercise 2.4

Topics Covered:

- Ansible Surveys



Automation controller surveys. **Adopt and grow.**



What is it?

- ▶ User-friendly, self-service interface in automation controller
- ▶ Abstracts complexity using question and answer format
- ▶ Best suited for teams directly accessing automation and close to the automation practice
- ▶ Access and execution governed using controller features

A screenshot of a web interface titled 'Prompt on Launch'. The interface has a red header bar with a menu icon on the left and window controls on the right. Below the header, there's a breadcrumb trail: 'Templates > Add user to groups'. The main content area is divided into two sections. On the left, there's a vertical list of steps: '1 Survey' (highlighted with a blue circle) and '2 Review'. On the right, there's a form titled 'WHICH GROUP(S) SHOULD INCLUDE THIS USER?' with a subtext '(Enter groups, one per line.) *'. Below the title is a large, empty text input field. At the bottom right of the form, there are three buttons: 'Next' (blue), 'Back' (grey), and 'Cancel' (blue).

Lab Time

Complete Exercise 2.4



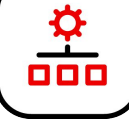
Exercise 2.5

Topics Covered:

- Role Based Access Control (RBAC)



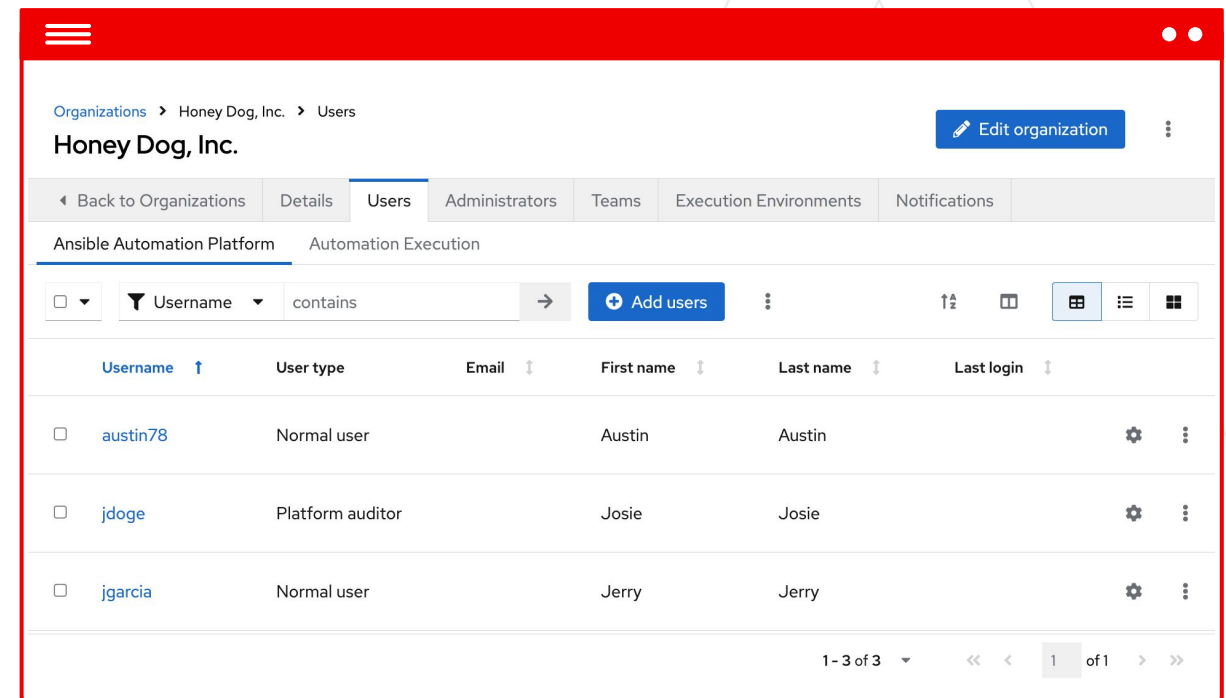
Role-Based Access Control. Who can use my automation?



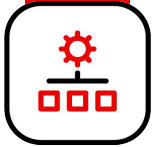
What is it?

Securely govern access to your automation

- ▶ Logically group controller objects and grant users and teams read, execute, edit permissions
- ▶ Use predefined roles to grant access
- ▶ Integrates with your existing enterprise authentication systems



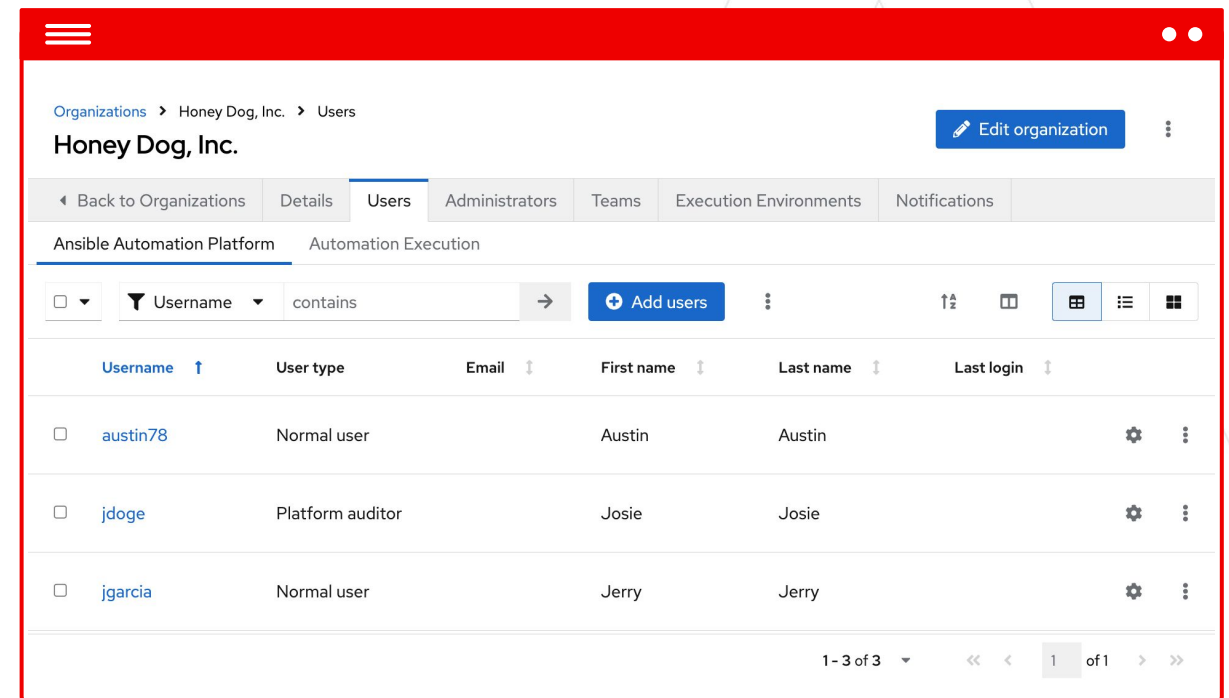
User Management



What is it?

Securely govern access to your automation

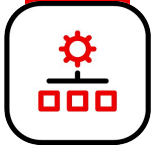
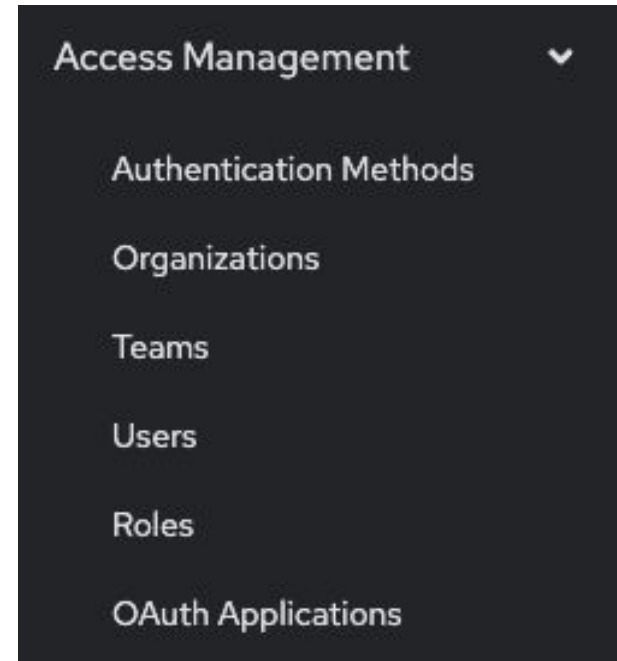
- ▶ Logically group controller objects and grant users and teams read, execute, edit permissions
- ▶ Use predefined roles to grant access
- ▶ Integrates with your existing enterprise authentication systems



User Management

Govern access to your automation

- ▶ An **organization** is a logical collection of users, teams, projects, inventories and more. All entities belong to an organization.
- ▶ A **user** is an account to access Ansible Automation Controller and its services given the permissions granted to it.
- ▶ **Teams** provide a means to implement role-based access control schemes and delegate responsibilities across organizations.



Lab Time

Complete Exercise 2.5



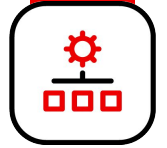
Exercise 2.6

Topics Covered:

- Workflows

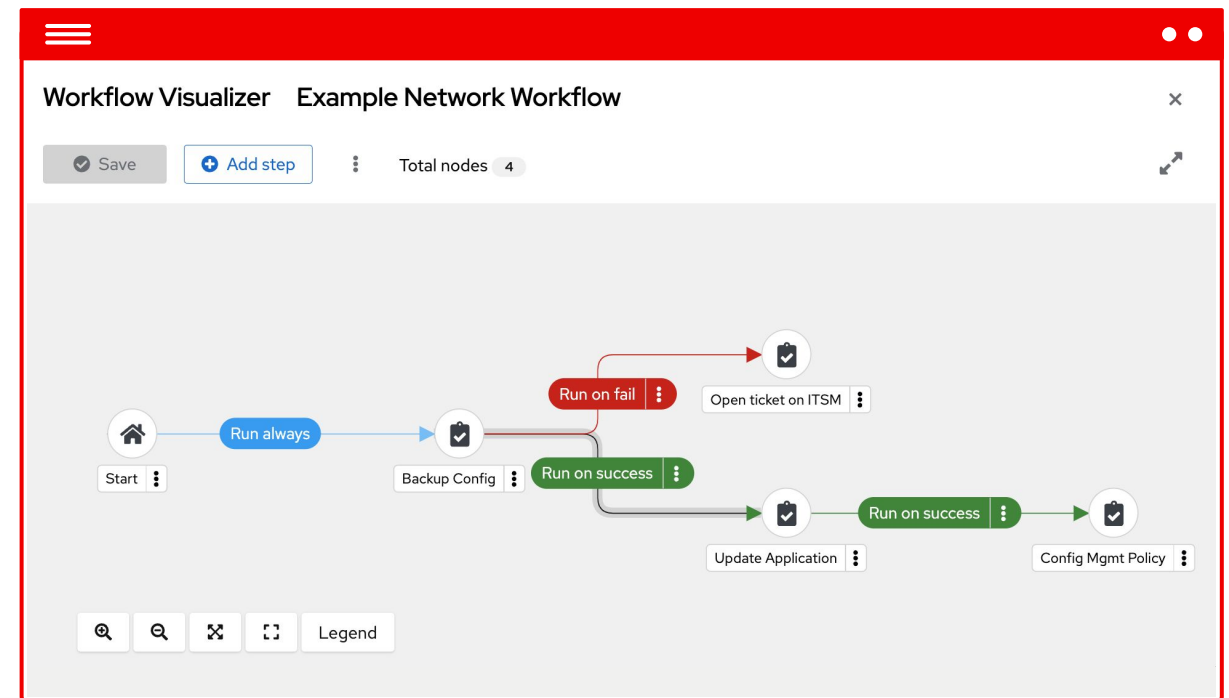


Workflows. Solve complex problems.



What is it?

- ▶ Overhauled in AAP 2.5
- ▶ Workflows enable the creation of powerful holistic automation, chaining together multiple pieces of automation and events
- ▶ Simple logic inside these workflows can trigger automation depending on the success or failure of previous steps
- ▶ Add approvals to your workflows to enhance governance
- ▶ Integrate other systems, such as ITSM to fit with your existing controls and processes



Lab Time

Complete Exercise 2.6



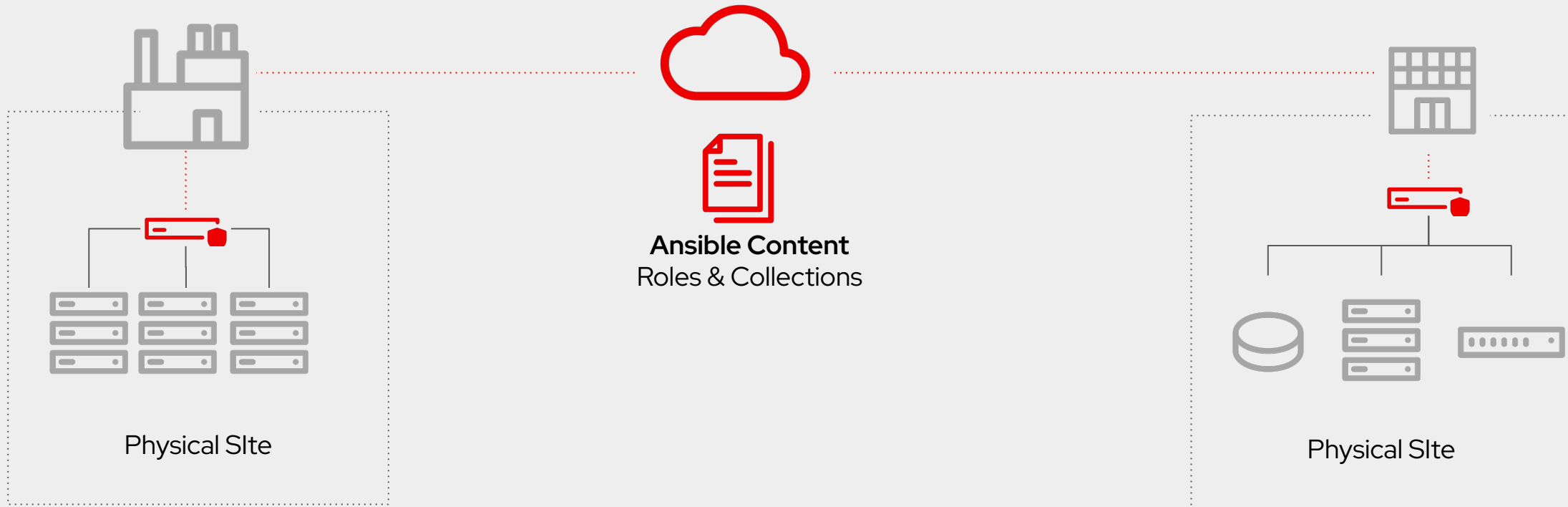
Exercise 2.7

Topics Covered:

- Red Hat Enterprise Linux System Roles



Automation Hub and Ansible Galaxy



Linux System Roles

- Consistent user interface to provide settings to a given subsystem that is abstract from any particular implementation

Examples



Email



kdump



network



selinux



timesync



firewall

An Ansible Playbook Variable Example

```
---
- name: example system roles playbook
  hosts: web

  tasks:

    - name: Configure Firewall
      include_role:
        name: linux-system-roles.firewall

    - name: Configure Timesync
      include_role:
        name: linux-system-roles.timesync
```

Lab Time

Complete Exercise 2.7



Exercise 2.8

Topics Covered:

- Wrap-Up



Lab Time

Complete Exercise 2.8





Where to go **next**



Learn more

- ▶ Workshops
- ▶ Documents
- ▶ Youtube
- ▶ X



Get started

- ▶ Self-paced labs
- ▶ Ansible Automation Platform trial
- ▶ console.redhat.com
- ▶ Ansible Lightspeed trial



Get serious

- ▶ Red Hat Automation Adoption Journey
- ▶ Red Hat Training
- ▶ Red Hat Consulting

Thank you

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.



linkedin.com/company/red-hat



youtube.com/c/AnsibleAutomation



facebook.com/redhatinc



twitter.com/ansible